

UNRAVELING THE COMPLEXITY OF MATH PROBLEMS WITH THE KMX DATASET

Justin Lo

Department of Computer Science, ETH Zürich
justlo@ethz.ch

ABSTRACT

Mathematical reasoning is a crucial aspect in evaluating human cognition and is a crucial part of many decision-making tasks. While current datasets focus on a narrow set of topics or question types, the Khan-Math-X (KMX) dataset is a comprehensive and diverse set of problems that is annotated with additional dimensions such as grade level, exercise names, and question types. This allows for more fine-grained analysis of model performance. We assess the performance of several models using various techniques and show that while certain methods improve performance across the board, their effectiveness varies significantly depending on the model and problem type, revealing the intricacies when dealing with different mathematical tasks. The KMX dataset enables a more nuanced understanding of model strengths and weaknesses, and offers a valuable resource for future research in mathematical reasoning.

1 INTRODUCTION

There has been an increased interest in language models over the past few years. Advancements in transformer models have allowed large language models to demonstrate exceptional capabilities in question-answering related tasks. However, mathematical reasoning remains a greater challenge for language models (Cobbe et al., 2021), often requiring specific tailored techniques to obtain reasonable performance. Mathematical reasoning is crucial in evaluating human cognition and is important in many decision-making tasks (Ahn et al., 2024). Hence, mathematical tasks such as arithmetic operations, logical deduction and problem solving, are important to improving the effectiveness of language models across various domains.

Previous studies have identified challenges in language models’ abilities in solving math word problems (Srivatsa & Kochmar, 2024). They highlight the difficulties of these models to handle lengthy questions with a wide variety of mathematical operations. Traditional datasets for solving math problems are very small, with the Dolphin (Shi et al., 2015) and Aggregate (Roy & Roth, 2017) datasets having only 1878 and 1492 problems respectively. In addition, they feature simple questions that follow a similar structure. Other existing benchmarks such as GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021) have made strides in improving these deficiencies, but often focus on specific problem types such as math word problems, which can lead to model overfitting on these tasks. SVAMP (Patel et al., 2021) modifies existing datasets to improve the linguistic variation, but it remains focused solely on elementary integer arithmetic questions. Other datasets (Amini et al., 2019; Koncel-Kedziorski et al., 2016) feature more math problems but do not provide the depth and diversity needed to fully assess a model’s performance across the full range of mathematical skills.

To address these limitations, we introduce the KMX (Khan-Math-X) dataset which consists of 3240 grade 1 to 8 math problems. This dataset provides a broader and more detailed evaluation of language models’ numerical reasoning capabilities (Appendix A). The dataset is sourced from the Khan Academy curriculum and has been annotated with dimensions such as grade level, topic, question type, exercise name and the complexity of the solution. This level of granularity allows researchers to explore model behavior in a more structured way, identifying specific areas where models excel and falter.

The KMX dataset also serves as a benchmark to investigate the efficacy of various modeling techniques, including fine-tuning (Parthasarathy et al., 2024; Jeong, 2024), self-consistency (Wang et al.,

Dataset	Scope	Dataset Size	Solution Format	Difficulty Categorization	Topic Categorization	Question Type Categorization	Additional Properties
ALG514	Algebra.com	514	Problem-Equation-Answer	✗	✗	✗	Linear Algebra with 28 unique equation system
Dolphin	Yahoo Answers	1878	Problem-Equation-Answer	✗	✗	✗	Number Word Problems
Aggregate	Varied	1492	Problem-Equation-Answer	✗	✗	✗	Simple Word Problems (+, −, ×, ÷)
GSM8K	Grade School Math	8500	Problem-Explanation-Answer	✗	✗	✗	
MATH	AMC 8 – AIME	12500	Problem-Explanation-Answer	✓ (Difficulty 1-5)	✓ (7 categories)	✗	Emphasizes advanced math concepts
ASDiv	Grade 1-6	2305	Problem-Equation-Answer	✓ (Grade 1-6)	✗	✓	Maximum 2 operators
SVAMP	Elementary Level	1000	Problem-Equation-Answer	✗	✗	✓ (4 categories)	Integer (+, −, ×, ÷) only
MAWPS	Varied	3320	Problem-Equation-Answer	✗	✗	✓ (6 categories)	Introduces a framework to construct datasets
MathQA	Math Word Problems based on AQuA	37200	Problem-Explanation-Equation-Answer	✗	✓	✗	Multiple-choice questions
CMATH	Grade 1-6 (Chinese)	1700	Problem-Answer	✓ (Grade 1-6)	✗	✗	Contains annotations of the number of reasoning steps and number of digits
KMX (Ours)	Grades 1-8	3240	Problem-Explanation-Answer	✓ (Grade 1-8)	✓ (18 categories)	✓ (9 categories)	Contains annotations on the lesson name and exercise name

Figure 1: Comparison of some current datasets with the KMX dataset. **Scope** defines the difficulty and range of problems covered. Some datasets come from online sources and do not have a well-defined range.

2023), and external solvers (Gao et al., 2023; He-Yueya et al., 2023). By examining model performance across different dimensions of math problems, we aim to provide a more comprehensive understanding of how language models learn and process mathematical reasoning tasks. We show the potential of using additional information from the added dimensions (e.g. exercise name) within the KMX dataset in helping language models to more effectively solve math problems.

The contributions of our work are as follows:

- We introduce a dataset that enables fine-grained analysis of model performance across a wide range of math problems.
- We highlight the complexity of mathematical reasoning and the potential for further research in improving model capabilities in this domain.
- We evaluate the impact of advanced modeling techniques on model accuracy, and show how simple methods using additional information from the dataset could be useful.

2 RELATED WORK

Related Datasets Several benchmarks have been developed to evaluate the mathematical reasoning capabilities of language models. Many older datasets are either too small, not suitable for testing with language models, or are not properly verified (Amini et al., 2019; Koncel-Kedziorski et al., 2016; Kushman et al., 2014; Ling et al., 2017; Roy & Roth, 2017; Saxton et al., 2019; Shi et al., 2015). Some datasets aim to improve the arithmetic accuracy of models by providing explanations as Python codes (Austin et al., 2021; Mishra et al., 2023; Toshniwal et al., 2024). However, this format is less useful for human understanding the output of the models. GSM8K (Cobbe et al., 2021) is a commonly used dataset for assessing language models’ problem-solving and reasoning abilities. However, the GSM8K dataset focuses only on math word problems and primarily involves integer arithmetic. This limits the scope of analysis that can be performed. Other datasets such as the ASDiv (Miao et al., 2021) dataset and the SVAMP (Patel et al., 2021) dataset target specific phrasings of Math Word problems but remain limited in diversity. The GHOSTS (Frieder et al., 2023), MATH (Hendrycks et al., 2021) and TheoremQA (Chen et al., 2023b) datasets aim to have more difficult questions that are challenging even for humans. However, they are less aligned to a standard education curriculum, and can prove to be too difficult for meaningful analysis. There have been several Chinese datasets introduced (Wang et al., 2017; Yang et al., 2023). In some of these datasets, extra annotations such as the grade, subject and number of reasoning steps are included to provide a more holistic dataset (Liu et al., 2024; Wei et al., 2023b; Wu et al., 2023). However, the language difference creates a different data distribution and is less applicable to our goals.

Knowledge Distillation Knowledge distillation has emerged as a critical technique for transferring the capabilities of large models to smaller, more efficient models (Hinton et al., 2015). By using a larger closed-source model to supervise the training of a smaller student model, knowledge distillation saves significant computational cost while retaining a high performance. In large language models, knowledge distillation is pivotal to leveraging the capabilities of state-of-the-art models such as GPT-4 (OpenAI et al., 2024) to help other models learn relevant information (Xu et al., 2024).

Prompting Prompting methods are a commonly used technique to improve the performance of language models. Prompting involves providing specific instructions to the model to guide its answer generation. In chain-of-thought prompting (Nye et al., 2021; Wei et al., 2023a), the model is instructed to “think step-by-step”. By being forced to break down complex tasks into smaller, more manageable parts, it has been empirically shown to improve the performance of language models. Another method that helps models implicitly learn how to respond is in-context learning (Brown et al., 2020). In addition to the question, some sample question-answer pairs are provided to the model as a demonstration. This enables the model to learn the pattern and produce answers of a similar nature as the provided in-context examples. However, models can be sensitive to the choice of in-context examples provided (Liu et al., 2021; Lu et al., 2023).

Fine-tuning Fine-tuning involves updating parameters of a pre-trained language model on a specific task or dataset, allowing the model to adapt the representations to better suit the desired target. Current focuses are on fine-tuning via the use of an enhanced dataset, which has been generated via augmentation or annotated in a specific manner (Liu et al., 2023; Yu et al., 2023; Zelikman et al., 2022). Fine-tuning is beneficial for more complex or domain-specific tasks such as in mathematical reasoning where pre-trained language models may lack sufficient exposure to the problem types during general training.

Self-consistency Self-consistency (Wang et al., 2023) is a technique where models sample a diverse set of reasoning paths, then select the most consistent answer by marginalizing out the sampled reasoning paths. Models outputs are generated at a higher temperature, increasing the variability of responses and then making use of a majority voting system to provide more accurate results. This helps to reduce errors in tasks where the model’s internal reasoning process may lead to multiple possible outputs.

External solvers A major weakness of language models is performing arithmetic calculations. Hence, external solvers can mitigate this issue by allowing language models to generate a logic flow, then performing the calculations externally. For instance, He-Yueya et al. (2023) use a symbolic solver which expresses quantities as variables and expressions, which are then solved to generate the final answer. Chen et al. (2023a) and Gao et al. (2023) use language models to generate Python solutions which are executed in math-based libraries such as SymPy (Meurer et al., 2017) to compute the answers. Recently, OpenAI has introduced GPT-4 Code Interpreter (Zhou et al., 2023) which allows the model to perform self-verification on its answer, reducing arithmetic mistakes.

3 THE KMX DATASET

While designing the KMX (Khan-Math-X) dataset, we adhere to some design principles that are aimed at targeting gaps in current datasets.

- **Alignment with Educational Standards:** The KMX dataset is designed to comprehensively represent real-world math problems sourced from the Khan Academy curriculum. Each problem is carefully annotated with grade, unit, lesson, and topic to ensure educational relevance and to facilitate fine-grained performance evaluation across a broad range of math skills.
- **Diversity and Depth for Robust Evaluation:** To support in-depth analysis, the dataset encompasses a variety of question types, including word problems and mathematical expressions, categorized into 18 distinct topics, and 9 question types. We strive to have a large distribution of math topics and question types available so that a more holistic understanding of models’ numerical reasoning capabilities can be accessed.
- **Smooth Progression for Model Evaluation:** The dataset is designed to accommodate a wide range of models, from the weakest to the most advanced. We aim for a smooth gra-

CMATH	
Problem:	(Translated) There are 9 cakes in the store. If 5 are sold, how many are left?
Answer:	4
Additional Fields	Grade: 1 #Steps: 1 #Digits: 1
Dolphin	
Problem:	one number is 11 more than another number. Find the two numbers if three times the larger exceeds four times the smaller number by 4.
Equation:	unkn: x,y", "equ: x=y+11", "equ: 3*x=4*y+4"
Answer:	{40; 29}
MathQA	
Problem:	average age of students of an adult school is 40 years . 120 new students whose average age is 32 years joined the school . as a result the average age is decreased by 4 years . find the number of students of the school after joining of the new students .
Options:	a) 1200 , b) 120 , c) 360 , d) 240 , e) none of these
Explanation:	let the original no . of students be x . according to situation , $40x + 120 * 32 = (x + 120) * 36$ $\Rightarrow 40x + 3840 = 36x + 4320$ $\Rightarrow 4x = 480$ $\Rightarrow x = 120$ so , required no . of students after joining the new students = $x + 120 = 240$.
Formula:	$\text{multiply}(\text{divide}(\text{subtract}(\text{multiply}(\text{add}(32, 4), 120), \text{multiply}(120, 32)), \text{subtract}(40, \text{add}(32, 4))), 4)$
Answer:	d
Additional Fields	Category: general
KMX (Ours)	
Problem:	Solve for t. $2(t+1) = 10$ t = ?
Explanation:	To solve for t in the equation $2(t+1) = 10$, follow these steps: 1. Distribute the 2: $2 * t + 2 * 1 = 2t + 2$. The equation becomes: $2t + 2 = 10$. 2. Subtract 2 from both sides: $2t + 2 - 2 = 10 - 2$. This simplifies to: $2t = 8$. 3. Divide by 2: $2t / 2 = 8 / 2$. This results in: $t = 4$. So, the solution is $t = 4$.
Answer:	4
Additional Fields	Grade: 7 Unit Name: Expressions, equations, & inequalities Lesson Name: Two-step equations intro Topic: linear-equations-and-functions Question Type: Expressions-with-Variables

Figure 2: Comparison of previous works’ dataset formats. Previous work with equations and formulae are difficult to process and do not provide insights into the reasoning ability of language models. The KMX dataset provides natural language explanations and other annotations that improve analysis of numerical reasoning problems.

dient in problem difficulty to ensure that high-performing models can excel, while lower-performing models still demonstrate meaningful progression. This approach provides researchers with a clear understanding of each model’s strengths and weaknesses, facilitating more precise benchmarking across different levels of model capabilities.

- **Ease of Use:** The KMX dataset is formatted to be readily usable by a wide range of models, with problems available in a standardized, human-readable format. Additionally, each problem is accompanied by natural language, step-by-step solutions, making it easier for both models and researchers to interpret the reasoning behind the answers.

Data Collection The KMX dataset consists of 3240 grade school math problems from Grade 1 to 8 (2285 training and 955 test). The dataset is adapted from the AMPS dataset presented by Hendrycks et al. (2021). In the AMPS dataset, the Khan Academy questions are only used to pre-train a model and little attention is provided to it. However, we believe that there are several key factors that encourage the analysis of the Khan Academy exercises as a isolated dataset. Firstly, KhanAcademy (2024) is a renowned educational platform which provides learning resources for people. These content are aligned with curriculum standards and educational objectives, providing clear indication of how to evaluate a model’s ability to solve math problems in an actual educational setting. Secondly, the Khan Academy dataset includes a variety of math topics and diverse question pool. This allows a more comprehensive evaluation of language models. Thirdly, as a platform curated for education, the questions and topics are sorted into differing difficulty levels, which allows a fine-grained analysis and understanding of a model’s performance. To enhance the usefulness of the dataset, various preprocessing steps and data annotations are applied as summarized in Figure 3.

Data Annotation Based on the exercise names provided in the dataset, we annotate each problem with its corresponding grade, unit name, and lesson name in alignment with the Khan Academy curriculum. This results in 34 unique unit names, 102 unique lesson names, and 149 unique exercise names.

To improve the categorization of topics and to facilitate more robust analysis, we manually consolidate the data into 18 distinct topics. The motivation for reducing the number of topics is that many unit names are closely related, leading to fragmented categories with too few problems per unit. By merging similar topics, we create broader categories that not only simplify the analysis but also ensure that each topic has a sufficient number of problems for meaningful evaluation.

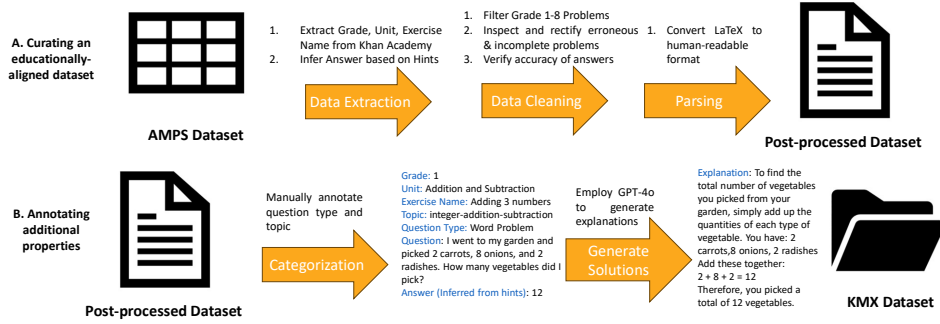


Figure 3: Creation of the KMX dataset

In addition, one key feature of the KMX dataset is the diversity of problems available. To highlight this and allow granular analysis, we annotate each question with 9 distinct question types. For instance, problems are classified into whether students are required to directly evaluate a mathematical expression, solve a word problem or write an expression as the answer. To enhance analysis within the test dataset, expressions and word problems are also labelled with the number of mathematical operations (e.g. $+$, $-$, $\times$, $\div$, exponents, roots) required to solve the question and the number of unique types of these operations. This information demonstrates the diversity of the KMX dataset and provides researchers an opportunity to better understand various models’ performances.

Generated Solutions The original Khan Academy dataset only consists of problems and hints. To provide a more complete dataset to assist in fine-tuning, we employ knowledge distillation via GPT-4o (Brown et al., 2020) by asking for a chain-of-thought-style solution (Wei et al., 2023a) based on the given problem and the last hint. Hence, we could extract a suitable answer and solution for the dataset.

Formatting Due to the variety of formatting present in the original data, we perform careful analysis to identify the various patterns used within the dataset. We convert problems from $\text{\LaTeX}$ notation to a human-readable format, removing any unnecessary commands that are used solely for styling purposes. This standardization is important as many language models are not directly trained on $\text{\LaTeX}$ notation and having such notation greatly impacts the tokenization and performance of the model.

Dataset Availability The full dataset can be found at <https://github.com/justlotw/KMX-dataset>. More details about the dataset can be found in Appendix A.

4 EXPERIMENTS

We explore the performance of a diverse range of language models on the KMX dataset. By incorporating models of varying sizes, we provide a comprehensive evaluation of how these models interact with complex math problems. This enables us to capture the nuances of model behavior and comprehend where models excel or falter in mathematical reasoning.

In our experiments, we conduct a progressive analysis of model performance. We first evaluate each model’s overall performance on the KMX dataset, and then conduct a more fine-grained analysis to investigate model behavior across specific problem categories. Lastly, we discuss how language model techniques affect the models’ ability to solve numerical reasoning problems. This three-tiered approach allows us to not only understand the models at a macro level, but also unravel the intricacies within math problems and better understand how we employ models to solve them.

4.1 EXPERIMENT SETUP

Model Selection In our experiments, we consider 17 language models (Table 1) to better understand how models of different capabilities would perform in the KMX dataset. The models are recent state-of-the-art models released between August 2023 and July 2024. We seek to perform a broad and representative analysis, focusing not only on state-of-the-art large language models, but

Table 1: List of models used and the number of parameters

MODEL	SIZE	MODEL	SIZE
DeepSeekMath-RL (Shao et al., 2024)	7B	Mistral (Jiang et al., 2023)	7B
Falcon (Almazrouei et al., 2023)	7B	Phi-2 (Abdin et al., 2024)	2.7B
Gemma (GemmaTeam et al., 2024)	2B	Qwen2 (Bai et al., 2023)	7B
GPT-3.5 (OpenAI et al., 2024)	-	Rho-Math (Lin et al., 2024)	1.1B
GPT-4o mini (OpenAI et al., 2024)	-	TinyLlama (Zhang et al., 2024)	1.1B
Llama 3 (Touvron et al., 2023)	8B	Vicuna (Zheng et al., 2023)	7B
Llama 3.1 (Touvron et al., 2023)	8B	Wizard-Math (Luo et al., 2023)	7B
Mathstral (MistralAI, 2024)	7B	Zephyr (Tunstall et al., 2023)	3B
MetaMath-Mistral (Yu et al., 2023)	7B		

also smaller language models which are more commercial-friendly and optimized for specific tasks. As a reference point for the possible upper bound of performance in numerical reasoning tasks, we use GPT3.5-turbo and GPT4o-mini (OpenAI et al., 2024). We choose a selection of 15 other models, aiming for a diverse set of models that can possess different capabilities in solving numerical reasoning problems. To ensure diversity in our analysis, we include models like Llama 3 (Touvron et al., 2023), Falcon (Almazrouei et al., 2023), and Qwen (Bai et al., 2023). Each of these models possess different architectures and a different selection of training datasets used, while having high performance on general natural language processing tasks. Models such as Phi-2 (Abdin et al., 2024) and Zephyr (Tunstall et al., 2023) are smaller models of around 3B parameters but perform well on traditional math reasoning benchmarks such as GSM8K. Additionally, we include DeepSeek-Math (Shao et al., 2024), Mathstral (MistralAI, 2024) and Wizard-Math (Luo et al., 2023) for their specific training on mathematical reasoning tasks, allowing us to assess the impact of specialized fine-tuning. These models have tailored training data and techniques and demonstrate state-of-the-art performances on current math benchmarks. This selection of models provide a broad spectrum of capabilities, from general NLP tasks to domain-specific models, allowing a robust evaluation of the different models on the KMX dataset.

Fairness and Dataset Integrity To maintain fairness in our evaluation, we closely examine the training data of each model, ensuring that none of the models had direct access to the exact question-answer pairs in the KMX dataset. While it is possible that models may have been exposed to similar content during pre-training, we find no significant data leakage from Khan Academy’s dataset in the commonly used pre-training corpora for these models. This precaution ensures that the models are being tested on unseen problems, allowing for an accurate assessment of their mathematical reasoning capabilities.

Model Evaluation As a baseline for comparison, we utilize In-Context Learning (ICL) with Chain-of-Thought (CoT) prompting. For each model, we design an 8-shot prompt to guide the model in producing answers in a consistent format. The CoT examples are carefully selected to cover a wide variety of grades and topics, ensuring that models have a diverse reference set. These examples are chosen to avoid biases toward any specific problem type and are provided to the models primarily as templates for generating solutions. For model generation, unless otherwise specified, we use the default Hugging Face generation parameters. The answers are extracted based on a specific format outlined in the prompt, and additional processing is performed to ensure consistency in the model outputs. To evaluate the correctness of the answers, a parsing mechanism checks whether the model-generated solutions matches the expected answers. Our evaluation metric is the percentage of correctly solved problems. To ensure that equivalent answers are graded fairly, we employ SymPy (Meurer et al., 2017) to simplify numerical expressions. This allows us to accurately evaluate cases where answers are mathematically equivalent but expressed differently (e.g. 0.5 is graded equivalently to $1/2$).

4.2 ANALYZING MODEL PERFORMANCE

Model Size We explore the overall accuracy of the models. Contrary to findings in previous research (Brown et al., 2020), we do not observe a clear trend correlating model size with accuracy. As shown in Figure 4, the GPT-3.5-turbo model, with an estimated 175B parameters, achieves an accuracy 87%. This is lower than the Qwen (89.8% accuracy) and DeepSeekMath (88.8% accuracy)

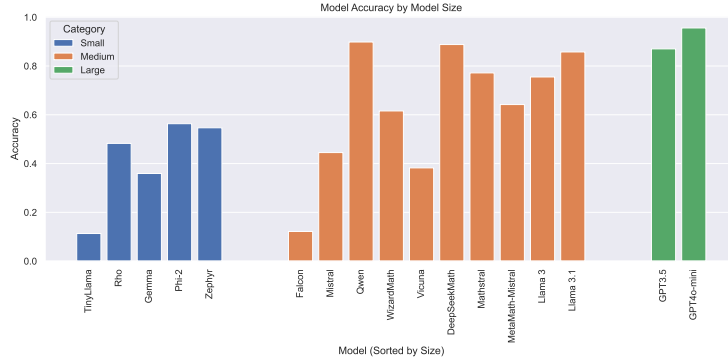


Figure 4: Accuracy of models on the KMX dataset, classified into small ($\leq 3B$ parameters), medium (7-8B parameters) and large models

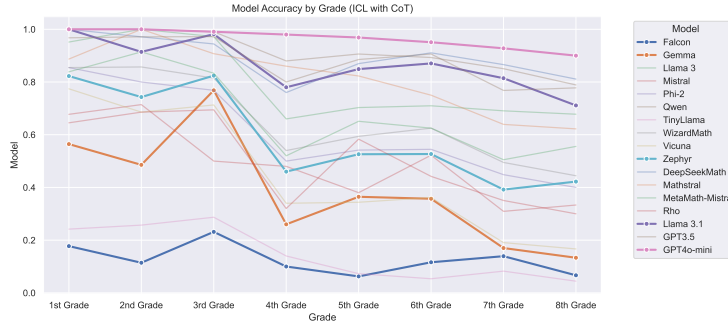


Figure 5: Model accuracy of the Math problems across different grades

models, which have over 20 times fewer parameters. This challenges the assumption that scaling model size is the default way to improve performance. There is significant variance within each model category, highlighting the complexity of solving math reasoning tasks and the importance of fine-tuning and task specialization. The KMX dataset captures these nuances, allowing for a more detailed investigation into model behavior.

Distribution of Dataset Problems To evaluate the usefulness of our dataset compared to traditional benchmarks, we compare the relative performance of models on the GSM8K dataset and the KMX dataset. While previous studies have shown that WizardMath outperforms MetaMath-Mistral in previous benchmarks, with both math models greatly outperforming GPT-3.5-turbo and Qwen, we observe that Qwen (89.8% accuracy) performs significantly better than WizardMath (61.6% accuracy) and MetaMath-Mistral (64.2% accuracy) in the KMX dataset, both of which are supposedly optimized for math reasoning tasks. This divergence hints at the possibility of math-specialized models overfitting towards traditional benchmarks. Our dataset captures a broader or different distribution of mathematical reasoning tasks, which may require models to have a more generalized understanding of math topics. The differences in performance ordering across benchmarks indicate that models which perform well on GSM8K may not necessarily generalize to other mathematical datasets, emphasizing the need for diverse evaluation datasets like KMX to gain a deeper understanding of model capabilities.

4.3 DIMENSIONS OF A MATH PROBLEM

Grade Level As an initial investigation, we analyze the impact of problem difficulty on model performance. This is done by comparing the models’ performances across different grade levels. As the grade level increases, we observe a general downward trend in the corresponding performance. This is in line with our expectation that the more advanced grades contain questions that are more challenging and may require more reasoning steps (Figure 5). However, we notice an unusual pattern that opposes the naive correlation between grade level and accuracy. In most models, there is an

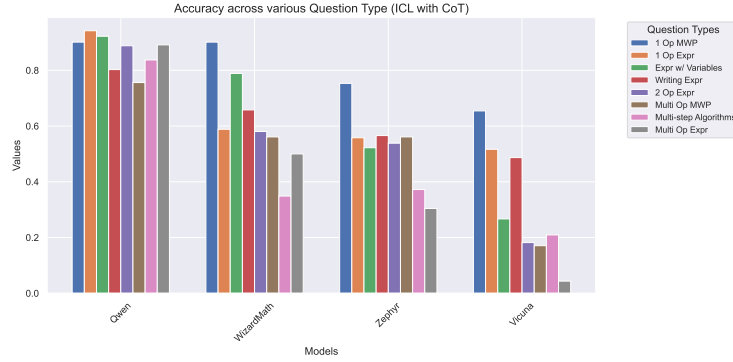


Figure 6: Accuracy of a few models across different question types

unexpected dip in performance in the 2nd and 4th grades, despite following the expected downward trajectory in later grades. This suggests that grade level alone is not a reliable indicator of a problem’s difficulty for language models, motivating an investigation into additional problem dimensions which could explain the variation in model performances.

Question Type To provide a more holistic understanding of math problems, we delve into how model accuracy varies across different question types. The results reveal that stronger models like Qwen (Figure 6) exhibit consistently high performance across most question types, while weaker models, such as Vicuna, show greater variability. There is a performance hierarchy where simpler question types such as math word problems and expressions with 1 type of operation (‘1 Op MWP’ and ‘1 Op Expr’ respectively) are generally solvable across all models. However, as the complexity of the question type increases (e.g. complex algorithms), performance drops significantly for weaker models. This demonstrates a natural progression of a model’s capabilities in solving mathematical questions which can be measured using the KMX dataset. A notable trend is that in 15 out of 17 models, ‘1 Op MWP’ outperforms ‘1 Op Expr’, suggesting potential overfitting to traditional math word datasets which often focus on word problems rather than raw expressions. This may indicate a dataset bias and highlights the need for datasets to cover a broader range of mathematical constructs.

Topic We also explore model performance across different mathematical topics. From the results (Table 2), we observe a trend on the difficulty of the topics. Questions focusing on integers are the easier for models to handle, while topics such as fractions and negative fractions pose greater challenges. This hierarchy of difficulty is accompanied by increasing variance in model performance. Models perform similarly well on easier topics but diverge significantly as the topic complexity increases. This suggests a deeper underlying structure in how models learn mathematical concepts, warranting further research into whether such a hierarchy is an inherent part of a model’s learning process. Of note, we observe that this behavior differs when applying different techniques and there are further nuances in explaining how each model reacts to different topics (Appendix C). Therefore, we also want to analyze various language model techniques and understand their impact on performance across the KMX dataset.

4.4 LANGUAGE MODEL TECHNIQUES

We use 10 models to analyze techniques that are used to improve language model performance. We explore how these methods affect model accuracy across question types and topics, demonstrating the flexibility of the KMX dataset in facilitating a more fine-grained understanding.

Fine-tuning Fine-tuning consistently improves performance across a wide range of models, even for models that initially struggled with in-context learning. Fine-tuning is important in more complex mathematical datasets like KMX which contain a different distribution of problems. We observe that each model’s relative performance to Qwen increases when using fine-tuning, bridging gaps in model learning across different mathematical domains and question types. This demonstrates the robustness of the KMX dataset as a benchmark for understanding and improving model learning across a variety of mathematical problems.

Table 2: Accuracy scores by topic per model. The topic mean and variance excludes Falcon as it struggles to adapt to the formats we provide, continually repeating the in-context learning prompts instead of generating an answer that is related to the problem. Coefficient of Variation (CV) measures the relative variation of the data around the mean, and is calculated by the standard deviation divided by the mean of the data.

Model	Integers (+-)	Integers (* /)	Decimals (+-)	Decimals (* /)	Fractions (+-)	Fractions (* /)	Decimal to Frac	Rates and %	Exponents and $\sqrt{}$	Variables & Expressions	Linear Eqs	Negative Numbers	Negative Fractions	Mean	Variance	Coeff of Variation (CV)
Falcon	0.125	0.242	0.115	0.055	0.000	0.048	0.263	0.080	0.148	0.111	0.068	0.231	0.086	0.121	0.007	0.666
Gemma	0.500	0.707	0.744	0.356	0.040	0.106	0.290	0.080	0.315	0.318	0.204	0.333	0.086	0.314	0.051	0.721
Llama 3	0.914	0.909	0.833	0.671	0.760	0.442	0.447	0.760	0.833	0.825	0.767	0.846	0.600	0.739	0.025	0.212
Mistral	0.594	0.485	0.500	0.315	0.240	0.327	0.421	0.200	0.556	0.556	0.427	0.487	0.257	0.413	0.017	0.319
Phi-2	0.703	0.768	0.641	0.425	0.440	0.346	0.553	0.400	0.685	0.730	0.524	0.718	0.143	0.544	0.035	0.342
Qwen	0.969	0.950	0.923	0.877	0.680	0.942	0.895	0.720	0.982	0.937	0.806	0.923	0.800	0.877	0.009	0.110
TinyLlama	0.219	0.273	0.103	0.055	0.000	0.019	0.237	0.040	0.111	0.079	0.049	0.103	0.057	0.103	0.008	0.837
WizardMath	0.719	0.788	0.590	0.356	0.520	0.577	0.421	0.600	0.815	0.730	0.631	0.667	0.371	0.599	0.022	0.250
Vicuna	0.711	0.606	0.654	0.151	0.000	0.212	0.342	0.200	0.185	0.587	0.214	0.333	0.086	0.329	0.055	0.712
Zephyr	0.711	0.758	0.436	0.288	0.440	0.596	0.316	0.520	0.722	0.667	0.476	0.513	0.229	0.513	0.030	0.336
DeepSeekMath	0.875	0.970	0.897	0.822	0.800	0.865	0.868	0.880	1.000	0.905	0.864	0.897	0.800	0.880	0.003	0.066
Mathstral	0.875	0.909	0.769	0.630	0.880	0.846	0.553	0.560	0.870	0.714	0.718	0.692	0.800	0.755	0.015	0.162
MetaMath-Mistral	0.734	0.778	0.795	0.384	0.360	0.673	0.395	0.680	0.648	0.619	0.699	0.718	0.286	0.598	0.031	0.294
Rho	0.609	0.626	0.603	0.370	0.560	0.519	0.263	0.280	0.611	0.429	0.437	0.667	0.143	0.471	0.027	0.352
Llama 3.1	0.930	0.960	0.910	0.712	0.880	0.779	0.579	0.800	0.944	0.873	0.903	0.846	0.829	0.842	0.011	0.126
GPT3.5	0.953	0.960	1.000	0.890	0.840	0.750	0.658	0.840	0.963	0.841	0.786	0.923	0.771	0.860	0.010	0.117
GPT4o-mini	0.992	0.970	1.000	0.877	0.960	0.981	0.790	0.880	1.000	0.905	0.961	1.000	1.000	0.947	0.004	0.069
MEAN*	0.751	0.776	0.712	0.511	0.525	0.561	0.502	0.528	0.703	0.670	0.592	0.667	0.454			
VARIANCE*	0.042	0.040	0.056	0.074	0.107	0.089	0.044	0.085	0.083	0.056	0.074	0.064	0.111			

Table 3: Accuracy of each model while employing the respective prompting/fine-tuning techniques
GD: greedy decoding generation; **SC**: generation with self-consistency
MEAN*/VARIANCE*: Mean/Variance of the 10 models also used in fine-tuning
MEAN/VARIANCE : Mean/Variance of all 17 models

	ICL without CoT		ICL with CoT		Fine-tuning		Calculator		Identifying Ex. Name	
	GD	SC	GD	SC	GD	SC	GD	SC	GD	SC
Falcon	0.207	0.184	0.122	0.116	0.314	0.236	0.140	0.249	0.320	0.341
Gemma	0.336	0.324	0.359	0.315	0.605	0.524	0.672	0.687	0.569	0.592
Llama 3	0.661	0.662	0.755	0.790	0.768	0.813	0.751	0.791	0.736	0.758
Mistral	0.465	0.461	0.445	0.447	0.606	0.640	0.270	0.429	0.635	0.664
Phi-2	0.538	0.523	0.563	0.589	0.769	0.744	0.744	0.688	0.759	0.773
Qwen	0.753	0.747	0.898	0.918	0.900	0.920	0.887	0.918	0.907	0.932
TinyLlama	0.187	0.161	0.113	0.104	0.198	0.165	0.406	0.390	0.221	0.198
WizardMath	0.583	0.692	0.616	0.722	0.782	0.791	0.816	0.814	0.780	0.833
Vicuna	0.134	0.225	0.382	0.440	0.466	0.494	0.595	0.676	0.425	0.494
Zephyr	0.441	0.440	0.547	0.602	0.625	0.607	0.639	0.646	0.599	0.629
DeepSeekMath	0.906	0.940	0.888	0.936	-	-	-	-	-	-
Mathstral	0.606	0.622	0.772	0.813	-	-	-	-	-	-
MetaMath-Mistral	0.583	0.673	0.642	0.720	-	-	-	-	-	-
Rho	0.335	0.334	0.483	0.416	-	-	-	-	-	-
Llama 3.1	0.730	0.764	0.858	0.892	-	-	-	-	-	-
GPT3.5	0.790	0.780	0.870	0.902	-	-	-	-	-	-
GPT4o-mini	0.793	0.783	0.956	0.948	-	-	-	-	-	-
MEAN*	0.431	0.442	0.480	0.504	0.603	0.593	0.592	0.629	0.595	0.621
VARIANCE*	0.044	0.046	0.063	0.075	0.049	0.061	0.060	0.044	0.047	0.051
MEAN	0.532	0.548	0.604	0.628	-	-	-	-	-	-
VARIANCE	0.054	0.057	0.069	0.078	-	-	-	-	-	-

Self-consistency In addition, we explore how self-consistency can aid a model’s performance in numerical reasoning. In previous work, self-consistency has been shown to be a generally superior method in tasks. However, our findings contradict these claims, showing that its effectiveness depends heavily on both the type of question and the model being used. For example, in solving ‘exponents and roots’ and ‘negative fraction arithmetic’, self-consistency performs worse than greedy decoding. Hence, the KMX dataset is crucial in providing a fine-grained analysis framework to explore model behaviors across different mathematical topics and question types.

Use of External Calculator External solvers are an option to mitigate arithmetic errors. We fine-tune models to recognize mathematical calculations, allowing SymPy to evaluate expressions during inference. Our experiments show that incorporating the external calculator result in an improvement in some question types but is less effective for math word problems with one type of operation and multi-step algorithms. This suggests that although an external calculator can assist in some calculations, it must be carefully tailored to the characteristics of the dataset and model employed.

Identifying the Exercise Name Additionally, we fine-tune models to first identify the exercise name before solving the problem, investigating whether an inclusion of this additional dimension from the KMX dataset offers additional insights. Our results demonstrate that this technique holds potential, particularly when combined with self-consistency. Excluding Llama 3, there is no statistical significance that models exhibit a decline in performance. For some complex topics, exercise identification provides a marked improvement in almost all models. These findings show that simple exercise-level information from the KMX dataset offer a promising avenue for improving model performances.

5 CONCLUSION

We present the KMX dataset which provides an opportunity for evaluating and improving language models’ mathematical reasoning abilities. Through our experiments, we demonstrate that there are many dimensions which affect the performance of a language model. We highlight the importance of using a diverse set of problems to fully assess the strengths and limitations of different models and techniques. We encourage the use of the KMX dataset to analyze models’ mathematical reasoning abilities in a more in-depth manner. Future work can investigate specific challenges and inspire new approaches to training models, leading to significant advancements in language models’ abilities to solve complex mathematical problems.

REFERENCES

- Marah Abdin, Jyoti Aneja, Sebastien Bubeck, Caio César Teodoro Mendes, Weizhu Chen, Allie Del Giorno, Ronen Eldan, Sivakanth Gopi, Suriya Gunasekar, Mojan Javaheripi, Piero Kauffmann, Yin Tat Lee, Yuanzhi Li, Anh Nguyen, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Michael Santacrose, Harkirat Singh Behl, Adam Taumann Kalai, Xin Wang, Rachel Ward, Philipp Witte, Cyril Zhang, and Yi Zhang. Phi-2: The surprising power of small language models. <https://mistral.ai/news/mathstral/>, 2024.
- Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. Large language models for mathematical reasoning: Progresses and challenges, 2024. URL <https://arxiv.org/abs/2402.00157>.
- Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, M  rouane Debbah,   tienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. The falcon series of open language models, 2023.
- Aida Amini, Saadia Gabriel, Peter Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. Mathqa: Towards interpretable math word problem solving with operation-based formalisms, 2019. URL <https://arxiv.org/abs/1905.13319>.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report, 2023.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. URL <https://arxiv.org/abs/2005.14165>.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks, 2023a.
- Wenhu Chen, Ming Yin, Max Ku, Pan Lu, Yixin Wan, Xueguang Ma, Jianyu Xu, Xinyi Wang, and Tony Xia. Theoremqa: A theorem-driven question answering dataset, 2023b. URL <https://arxiv.org/abs/2305.12524>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit optimizers via block-wise quantization, 2022. URL <https://arxiv.org/abs/2110.02861>.
- Simon Frieder, Luca Pinchetti, Alexis Chevalier, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Christian Petersen, and Julius Berner. Mathematical capabilities of chatgpt, 2023. URL <https://arxiv.org/abs/2301.13867>.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models, 2023.

- GemmaTeam, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussonot, Pier Giuseppe Sessa, Aakanksha Chowdhery, Adam Roberts, Aditya Barua, Alex Botev, Alex Castro-Ros, Ambrose Slone, Amélie Héliou, Andrea Tacchetti, Anna Bulanova, Antonia Paterson, Beth Tsai, Bobak Shahriari, Charline Le Lan, Christopher A. Choquette-Choo, Clément Crepy, Daniel Cer, Daphne Ippolito, David Reid, Elena Buchatskaya, Eric Ni, Eric Noland, Geng Yan, George Tucker, George-Christian Muraru, Grigory Rozhdestvenskiy, Henryk Michalewski, Ian Tenney, Ivan Grishchenko, Jacob Austin, James Keeling, Jane Labanowski, Jean-Baptiste Lespiau, Jeff Stanway, Jenny Brennan, Jeremy Chen, Johan Ferret, Justin Chiu, Justin Mao-Jones, Katherine Lee, Kathy Yu, Katie Millican, Lars Lowe Sjoesund, Lisa Lee, Lucas Dixon, Machel Reid, Maciej Mikula, Mateo Wirth, Michael Sharman, Nikolai Chinaev, Nithum Thain, Olivier Bachem, Oscar Chang, Oscar Wahltinez, Paige Bailey, Paul Michel, Petko Yotov, Rahma Chaabouni, Ramona Comanescu, Reena Jana, Rohan Anil, Ross McIlroy, Ruibo Liu, Ryan Mullins, Samuel L Smith, Sebastian Borgeaud, Sertan Girgin, Sholto Douglas, Shree Pandya, Siamak Shakeri, Soham De, Ted Klimentko, Tom Hennigan, Vlad Feinberg, Wojciech Stokowiec, Yu hui Chen, Zafarali Ahmed, Zhitao Gong, Tris Warkentin, Ludovic Peran, Minh Giang, Clément Farabet, Oriol Vinyals, Jeff Dean, Koray Kavukcuoglu, Demis Hassabis, Zoubin Ghahramani, Douglas Eck, Joelle Barral, Fernando Pereira, Eli Collins, Armand Joulin, Noah Fiedel, Evan Senter, Alek Andreev, and Kathleen Kenealy. Gemma: Open models based on gemini research and technology, 2024.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017. URL <https://arxiv.org/abs/1706.04599>.
- Joy He-Yueya, Gabriel Poesia, Rose E. Wang, and Noah D. Goodman. Solving math word problems by combining language models with symbolic solvers, 2023.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015. URL <https://arxiv.org/abs/1503.02531>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Xiaoyang Hu, Shane Storks, Richard L. Lewis, and Joyce Chai. In-context analogical reasoning with pre-trained language models, 2023. URL <https://arxiv.org/abs/2305.17626>.
- Cheonsu Jeong. Domain-specialized llm: Financial fine-tuning and utilization method using mistral 7b. *Journal of Intelligence and Information Systems*, 30(1):93–120, March 2024. ISSN 2288-4882. doi: 10.13088/jiis.2024.30.1.093. URL <http://dx.doi.org/10.13088/jiis.2024.30.1.093>.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023.
- KhanAcademy, 2024. URL <https://www.khanacademy.org/>.
- Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. MAWPS: A math word problem repository. In Kevin Knight, Ani Nenkova, and Owen Rambow (eds.), *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 1152–1157, San Diego, California, June 2016. Association for Computational Linguistics. doi: 10.18653/v1/N16-1136. URL <https://aclanthology.org/N16-1136>.
- Nate Kushman, Yoav Artzi, Luke Zettlemoyer, and Regina Barzilay. Learning to automatically solve algebra word problems. In Kristina Toutanova and Hua Wu (eds.), *Proceedings of the*

- 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 271–281, Baltimore, Maryland, June 2014. Association for Computational Linguistics. doi: 10.3115/v1/P14-1026. URL <https://aclanthology.org/P14-1026>.
- Zhenghao Lin, Zhibin Gou, Yeyun Gong, Xiao Liu, Yelong Shen, Ruochen Xu, Chen Lin, Yujiu Yang, Jian Jiao, Nan Duan, and Weizhu Chen. Rho-1: Not all tokens are what you need, 2024. URL <https://arxiv.org/abs/2404.07965>.
- Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. Program induction by rationale generation : Learning to solve and explain algebraic word problems, 2017. URL <https://arxiv.org/abs/1705.04146>.
- Bingbin Liu, Sebastien Bubeck, Ronen Eldan, Janardhan Kulkarni, Yuanzhi Li, Anh Nguyen, Rachel Ward, and Yi Zhang. Tinygsm: achieving ϵ_{80}
- Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for gpt-3?, 2021.
- Wentao Liu, Qianjun Pan, Yi Zhang, Zhuo Liu, Ji Wu, Jie Zhou, Aimin Zhou, Qin Chen, Bo Jiang, and Liang He. Cmm-math: A chinese multimodal math dataset to evaluate and enhance the mathematics reasoning of large multimodal models, 2024. URL <https://arxiv.org/abs/2409.02834>.
- Pan Lu, Liang Qiu, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, Tanmay Rajpurohit, Peter Clark, and Ashwin Kalyan. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning, 2023.
- Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct, 2023.
- Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. Sympy: symbolic computing in python. *PeerJ Computer Science*, 3:e103, January 2017. ISSN 2376-5992. doi: 10.7717/peerj-cs.103. URL <https://doi.org/10.7717/peerj-cs.103>.
- Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing english math word problem solvers, 2021. URL <https://arxiv.org/abs/2106.15772>.
- Swaroop Mishra, Matthew Finlayson, Pan Lu, Leonard Tang, Sean Welleck, Chitta Baral, Tanmay Rajpurohit, Oyvind Tafjord, Ashish Sabharwal, Peter Clark, and Ashwin Kalyan. Lila: A unified benchmark for mathematical reasoning, 2023. URL <https://arxiv.org/abs/2210.17517>.
- MistralAI. Mathstral. <https://mistral.ai/news/mathstral/>, 2024.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2021.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey

Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeesh Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.

Venkatesh Balavadhani Parthasarathy, Ahtsham Zafar, Aafaq Khan, and Arsalan Shahid. The ultimate guide to fine-tuning llms from basics to breakthroughs: An exhaustive review of technologies, research, best practices, applied research challenges and opportunities, 2024. URL <https://arxiv.org/abs/2408.13296>.

Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. Openwebmath: An open dataset of high-quality mathematical web text, 2023. URL <https://arxiv.org/abs/2310.06786>.

Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are nlp models really able to solve simple math word problems?, 2021. URL <https://arxiv.org/abs/2103.07191>.

Subhro Roy and Dan Roth. Mapping to declarative knowledge for word problem solving, 2017. URL <https://arxiv.org/abs/1712.09391>.

David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models, 2019. URL <https://arxiv.org/abs/1904.01557>.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.
- Shuming Shi, Yuehui Wang, Chin-Yew Lin, Xiaojiang Liu, and Yong Rui. Automatically solving number word problems by semantic parsing and reasoning. In Lluís Màrquez, Chris Callison-Burch, and Jian Su (eds.), *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 1132–1142, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1135. URL <https://aclanthology.org/D15-1135>.
- KV Aditya Srivatsa and Ekaterina Kochmar. What makes math word problems challenging for llms?, 2024. URL <https://arxiv.org/abs/2403.11369>.
- Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. Openmathinstruct-1: A 1.8 million math instruction tuning dataset, 2024. URL <https://arxiv.org/abs/2402.10176>.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro von Werra, Cl  mentine Fourrier, Nathan Habib, Nathan Sarrazin, Omar Sanseviero, Alexander M. Rush, and Thomas Wolf. Zephyr: Direct distillation of lm alignment, 2023.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023.
- Yan Wang, Xiaojiang Liu, and Shuming Shi. Deep neural solver for math word problems. In Martha Palmer, Rebecca Hwa, and Sebastian Riedel (eds.), *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pp. 845–854, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi: 10.18653/v1/D17-1088. URL <https://aclanthology.org/D17-1088>.
- Taylor Webb, Keith J. Holyoak, and Hongjing Lu. Emergent analogical reasoning in large language models, 2023. URL <https://arxiv.org/abs/2212.09196>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023a.
- Tianwen Wei, Jian Luan, Wei Liu, Shuang Dong, and Bin Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023b. URL <https://arxiv.org/abs/2306.16636>.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, R  mi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Huggingface’s transformers: State-of-the-art natural language processing, 2020.

- Haoyi Wu, Wenyang Hui, Yezeng Chen, Weiqi Wu, Kewei Tu, and Yi Zhou. Conic10k: A challenging math problem understanding and reasoning dataset, 2023. URL <https://arxiv.org/abs/2311.05113>.
- Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. A survey on knowledge distillation of large language models, 2024. URL <https://arxiv.org/abs/2402.13116>.
- Zhen Yang, Ming Ding, Qingsong Lv, Zhihuan Jiang, Zehai He, Yuyi Guo, Jinfeng Bai, and Jie Tang. Gpt can solve mathematical problems without a calculator, 2023. URL <https://arxiv.org/abs/2309.03241>.
- Michihiro Yasunaga, Xinyun Chen, Yujia Li, Panupong Pasupat, Jure Leskovec, Percy Liang, Ed H. Chi, and Denny Zhou. Large language models as analogical reasoners, 2024. URL <https://arxiv.org/abs/2310.01714>.
- Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models, 2023.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022.
- Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model, 2024. URL <https://arxiv.org/abs/2401.02385>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL <https://arxiv.org/abs/2306.05685>.
- Aojun Zhou, Ke Wang, Zimu Lu, Weikang Shi, Sichun Luo, Zipeng Qin, Shaoqing Lu, Anya Jia, Linqi Song, Mingjie Zhan, and Hongsheng Li. Solving challenging math word problems using gpt-4 code interpreter with code-based self-verification, 2023.

A ADDITIONAL DATASET DETAILS

Grade	Problem	Answer	Generated Explanation	Topic	Question Type
1	The zoo got 18 new animals. 3 of them are lions and the rest are monkeys. How many new monkeys did the zoo get?	15	The zoo got 18 new animals, and 3 of them are lions. Thus, the number of new monkeys is $18 - 3 = 15$.	integer-addition-subtraction	Word Problem
3	What is the value of j ? $24 = j * 3$ $j = ?$	8	To find the value of j , you need to solve for j in the equation $24 = j * 3$. To isolate j , you can divide both sides of the equation by 3: $24 / 3 = j$. Simplifying, you get $j = 8$. Therefore, the value of j is 8.	integer-multiplication-division	Expression
4	Write your answer as a whole number and remainder. $5 / 2 = ?$	2 R 1	When you divide 5 by 2, it goes in 2 times because $2 * 2 = 4$. After subtracting 4 from 5, you have a remainder of 1. So, the result is 2 remainder 1.	integer-multiplication-division	Quotient-Remainder
6	Camelle manages her soccer team. She needs to order 4 practice jerseys for each player. There are y players. How many total practice jerseys does she need to order? Write your answer as an expression.	$4y$	Camelle needs to order 4 practice jerseys for each player. Since there are y players, she needs to order 4 jerseys for each player, which gives a total of $4y$ jerseys.	variables-and-expressions	Write-Expression
8	The sum of 5 consecutive integers is 505. What is the third number in this sequence?	101	Let the five consecutive integers be $x, x+1, x+2, x+3$, and $x+4$. The sum of these integers is $x + (x+1) + (x+2) + (x+3) + (x+4) = 5x + 10 = 505$. Solving for x , we get $x = 99$. The third number in the sequence is $x + 2 = 99 + 2 = 101$.	linear-equations-and-functions	Multi-step Algorithm

Figure 7: Sample problems, with a subset of the added annotations. These problems show an example of the different types of questions in the KMX Dataset

A.1 DATASET PARSING

Due to the variety of formatting present in the original data, we perform careful analysis to identify the various patterns used within the dataset. We convert problems from $\text{\LaTeX}$ notation to a human-readable format, removing any unnecessary commands that are used solely for styling purposes. This standardization is important as many language models are not directly trained on $\text{\LaTeX}$ notation and having such notation greatly impacts the tokenization and performance of the model.

As an initial step, we thoroughly review the dataset and manually edit questions that are ambiguous or that would result in difficulties with a parser. For instance, a question such as

```
Evaluate \u2212 216 \u2212 \u2212 \u2212 \u2212 \u2212 \u221a 3
\\sqrt[\\leftroot{-2}\\uproot{2}\\scriptstyle 3]{}{-216}.
```

is first edited to be

```
Evaluate \\sqrt[3]{}{-216}.
```

Then, we design a parser that replaces the $\text{\LaTeX}$ formatting with a suitable equivalent expression. The original dataset contains many $\text{\LaTeX}$ formatting meant for visual styling of questions (such as vertical calculation formatting) and hence are removed in our pre-processing. Table 4 contains a small subset of the changes we apply. Several problems contained mixed fractions (e.g. $1(2/3)$). To prevent ambiguity, we include the appropriate operation to prevent the model from interpreting it as a multiplication between two numbers (e.g. $-5(2/3)$ is changed to $-5 - (2/3)$).

Table 4: Sample pattern replacements performed. Any empty cells indicate that the pattern was removed without replacement.

Pattern	Replacement
$\backslash cdot$	*
$\backslash \dfrac{a}{b}$	$((a)/(b))$
$\backslash div$	/
$\backslash geq$	$\geq$
$\backslash left$	
$\backslash overline{x}$	x , where x is recurring
$\backslash phantom{x}$	
$\backslash underline$	

To ensure a more robust test dataset, we remove the multiple choice questions because they are inconsistent in their output format and contain many duplicate questions, with just their multiple choice options being permuted. Hence, we remove any multiple choice options and delete any resulting duplicates to reduce redundancies within the dataset.

A.2 DATA ANNOTATION

One of the key features of the KMX dataset is the dimensionality present, which enables in-depth analysis. To help with the annotation of these features, a few crucial steps are performed.

Feature	Value
Number of Problems	3240 (2285 train, 955 test)
Unique Grades	8
Unique Unit Names	34
Unique Lesson Names	102
Unique Exercise Names	149
Unique Topics*	18
Unique Question Types*	9

Table 5: Key dataset statistics of KMX, where * indicates a manually annotated fields

Firstly, using the exercise names provided in the original dataset, we use the KhanAcademy (2024) website to obtain information about the grade, unit name and lesson name that each problem belongs to. This divides the problems from grade 1 to 8, which contains 34 unit names, 102 lesson names and 149 exercise names. A summary of the dataset statistics is available in Table 5.

To improve the categorization of topics and to facilitate more robust analysis, we manually consolidate the data into 18 distinct topics, using the previously extracted information as a guide. The motivation for reducing the number of topics is that many unit names are closely related, leading to fragmented categories with too few problems per unit. By merging similar topics, we create broader categories that not only simplify the analysis but also ensure that each topic has a sufficient number of problems for meaningful evaluation. These topics cover a diverse pool of mathematical concepts, ranging from basic arithmetic for integers to more complicated ideas such as variables within expressions and specialized knowledge such as scientific notations. Each question in the test dataset is manually annotated with a topic. We then compile a mapping from a tuple of unit name, lesson name and exercise name to be mapped to the newly created topics. This allows us to use this mapping to assign the training dataset with their respective topics. In the occasional case where there is not a unique mapping, we assign training set problems with the most frequent mapping for that key.

In addition, one key feature of the KMX dataset is the diversity of problems available. To highlight this and allow granular analysis, we annotate each question into 9 distinct question types. For instance, problems are classified into whether the student is requested to directly evaluate a mathematical expression, solve a word problem or write an expression as the answer. We repeat the procedure that we use to create the mapping for the topic names, helping us identify 9 different question types in the test and training datasets. A full list of the annotated topics and question types can be found in Figure 8.

To demonstrate the potential of the dataset, we list the expressions required to solve each question within the test dataset. This enables us to count the number of mathematical operations (e.g. $+$, $-$, $\times$, $\div$, exponents, roots) necessary to solve each question, and the number of distinct mathematical operations present. This serves as a proxy for the complexity of the problem, and enables further analysis that researchers could conduct. In our experiments, we use this information to split the ‘expressions’ question type into those that require 1 unique type of operation, 2 unique types of operations, and more than 2 unique types operations. Similarly, the ‘word problems’ are split into those that require 1 unique type of operation and those that require more than 1 unique type of operation.

This information demonstrates the diversity of the KMX dataset and sets the KMX dataset apart from existing datasets. We provide an educationally-aligned set of problems that are annotated in detail and provide researchers an opportunity to better understand various models’ performances.

List of Topics:	
'integer-addition-subtraction', 'integer-multiplication-division', 'integer-multi-operations', 'fraction-multiplication-division', 'convert-decimal-fraction', 'external-topics', 'decimal-addition-subtraction', 'fraction-addition-subtraction', 'decimal-multiplication-division', 'rates-and-percentages', 'exponents-and-roots', 'variables-and-expressions', 'factors', 'linear-equations-and-functions', 'negative-numbers-arithmetic', 'negative-fractions-arithmetic', 'rational-numbers-addition-subtraction', 'scientific-notation'	
List of Question Types:	
'Expression', 'Word Problem', 'Quotient-Remainder', 'Convert Decimal Fraction', 'Write-Expression', 'Multi-step Algorithm', 'Expressions-with-Variables', 'Word-Problems-with-Variables', 'Convert-Scientific-Notation'	

Figure 8: List of topics and question types in the KMX dataset

A.3 GENERATION OF SOLUTIONS

System:	You are an AI tutor who provides explanations for Math questions. Do not use any LaTeX formatting.
User:	I have the following question: {question} The answer is: {answer}
Assistant:	The answer is correct. Here is the explanation without any LaTeX formatting:

Figure 9: Prompt to generate solutions from GPT3.5-turbo

The original Khan Academy dataset only consists of problems and hints. To provide a more complete dataset to assist in fine-tuning, we employ knowledge distillation via GPT3.5-turbo (Brown et al., 2020) to leverage its capacity to provide detailed responses. Since the original dataset does not directly contain the answers, we extract the final statement of the hints and use that as the answer. By structuring a prompt that contains both the original math question and the extracted answer (Figure 9), we generate explanations with a chain-of-thought (Wei et al., 2023a) reasoning. This serves as a distillation of knowledge from the large GPT-4o model to be incorporated into the KMX dataset so that it can be passed on while training other open-source models. The inclusion of these explanations enhances the dataset’s utility for both model training and evaluation, and is crucial for understanding and interpreting models’ mathematical reasoning.

A.4 DATASET SUMMARY

Table 6: Fields present in the KMX dataset

Dataset Columns	Description	Source	Additional Details
Problem	Math Question	Post-processed after extraction from the original AMPS dataset	Format has been standardized and ambiguous problems and answers changed / removed
Answer	Number / Expression / Equation answer	Extracted from the list of hints within the original dataset	
Solution	Step-by-step solution to arriving at the answer	GPT-4o	Prompted to generate chain-of-thought reasoning based on the Problem and Answer
Grade	Grade level (ranging from 1 st grade to 8 th grade)	Extracted from Khan Academy website	Utilize the Exercise Name to find which Grade, Unit and Lesson the problem has been classified into within Khan Academy.
Unit Name	Unit Name within Khan Academy		
Lesson Name	Lesson Name within Khan Academy	Folder names within the original dataset	The exercises are categorized in the following ordering: Grades > Units > Lessons > Exercises
Exercise Name	Exercise Name within Khan Academy		
Topic	Mathematical topics (e.g. Integer addition/subtraction, Exponents and roots)	Manual Annotation	Annotated on the test dataset based on review of each question, and their assigned unit/lesson/exercise name.
Question Type	Type of question (e.g. Equation / Word Problem)	Manual Annotation	A mapping is derived from the test dataset, which is then used to annotate the training dataset.
Num Op	Number of Math operations (+ $- \times \div e^? \sqrt{?}$) required to solve the problem	Manual Annotation	Annotated for the test dataset for problems classified as ‘Expressions’ or ‘Word Problems’. This is determined via the expression(s) required to solve the question
Num Unique Op	Number of distinct types of Math operations required to solve the problem	Manual Annotation	
Integer Answer	Whether the final answer is an integer	Manual Annotation	Assists evaluation of results
Split	Whether the data belongs to the train / test dataset	Manual Annotation	Data is stratified based on Exercise Name

B EXPERIMENT DETAILS

B.1 DATA LEAKAGE

Since the data is obtained as a subset of the AMPS (Hendrycks et al., 2021) dataset, we recognise the possibility that some of the models may have been exposed to parts of the KMX dataset. To maintain fairness in our evaluation, we closely examine the training data of each model, ensuring that none of the models had direct access to the exact question-answer pairs in the KMX dataset. In some of the models such as Gemma (GemmaTeam et al., 2024), Mathstral (MistralAI, 2024) and Qwen (Bai et al., 2023), the datasets used for training the models are either not stated or are vague, providing little information on possible data overlap.

However, in all the other models, the most frequent datasets used are Common Crawl¹ and Open-Web-Math (Paster et al., 2023). We observe that the Khan Academy domain exists within the Common Crawl repository. However, while inspecting the Khan Academy website, we note that the questions and hints cannot be parsed as they have been hidden from the source code and instead reference a separate system. This is further verified by analyzing the Open-Web-Math dataset, which is a subset of mathematical web pages from Common Crawl. While looking at the Open-Web-Math dataset, we observe the existence of Khan Academy domains. However, these are mostly forum posts, or in some cases, there are a few problems but without any corresponding hints or solutions. To further confirm the lack of intersection between our dataset and the model training, we search for some of the KMX dataset problems within the Open-Web-Math dataset but do not observe any overlaps.

While it is possible that models may have been exposed to similar content during pre-training, our investigation does not show any significant data leakage from Khan Academy’s dataset in the commonly used pre-training corpora for these models. We are confident that any exposure is minimal and is unlikely to significantly affect the findings presented. This precaution ensures that the models are being tested on unseen problems, allowing for an accurate assessment of their mathematical reasoning capabilities.

B.2 LIST OF EXPERIMENTS

All experiments are performed on the ETH Euler cluster using GPUs of 24GB memory, and are designed to be conducted within this resource constraint. For each experiment, we design a prompt template that can be input into Hugging Face (Wolf et al., 2020) as a user-assistant conversation. Then, we utilize each model’s chat template to convert the prompt template into a format that the models are trained on.

A total of 12 sets of experiments are conducted, representing 6 different techniques, once using greedy decoding (GD) and another using self-consistency (SC). We describe the 6 different techniques below. We henceforth refer to the experiments using their respective numbers (e.g. Experiment 3 refers to the experiment on fine-tuning).

1. **In-context learning without chain-of-thought reasoning (ICL without CoT)** In this technique, we use the existing models and provide an 8-shot in-context learning before prompting the model with the question. This does not require additional fine-tuning and can be applied to the model directly. Each example is provided using a Question-Answer format (Figure 10).
2. **In-context learning with chain-of-thought reasoning (ICL with CoT)** This is similar to the above but also includes a step-by-step explanation before providing the final answer. Each example is provided using a Question-Explanation-Answer format.
3. **Fine-tuning** We perform fine-tuning via the Question-Explanation-Answer format. The training data is provided to the models and trained for several epochs. After training, we prompt the fine-tuned models with the question only.
4. **Fine-tuning with calculator annotations (Calculator)** In a similar way as fine-tuning (Experiment 3), we use the training dataset to provide Question-Explanation-Answer data to the models. However, the explanations are substituted with a modified set

¹<https://commoncrawl.org/>

Table 7: List of models that are used for fine-tuning. Models marked with * are trained using LoRA

Model	Size	Model	Size
Falcon* Almazrouei et al. (2023)	7B	Qwen2* Bai et al. (2023)	7B
Gemma GemmaTeam et al. (2024)	2B	TinyLlama Zhang et al. (2024)	1.1B
Llama 3* Touvron et al. (2023)	8B	Vicuna* Zheng et al. (2023)	7B
Mistral* Jiang et al. (2023)	7B	Wizard-Math* Luo et al. (2023)	7B
Phi-2 Abdin et al. (2024)	2.7B	Zephyr Tunstall et al. (2023)	3B

of annotations. Using GPT-4o, we generate explanations where specific calculator annotations are inserted to identify arithmetic expressions. This annotation allows the integration of external solvers for computations, improving handling of arithmetic-heavy questions. More details about the format and usage of these annotations can be found in Section B.5.

5. **Pre-train on additional dataset consisting of calculator annotations, then fine-tune with calculator annotations (Calculator with pre-training)** This approach involves pre-training the models on a larger dataset specifically generated to teach the calculator tagging format. We use GPT-4o to generate a dataset of approximately 10000 questions with appropriate tagging for arithmetic expressions. The models are pre-trained using this dataset to familiarize them with the annotation format, before proceeding to the methodology employed for fine-tuning with the calculator annotations (Experiment 4).
6. **Fine-tuning with identification of exercise name (ID Exercise)** In this method, an additional line stating the name of the exercise is included in the explanations (e.g., “The topic is: [exercise name]”). The model is fine-tuned with this additional task of identifying the exercise topic before proceeding to generate the explanation. Although we do not evaluate the accuracy of the classification as a metric, it serves as a useful prompt to guide the model’s reasoning process towards the correct answering pathway.

B.3 MODEL TRAINING AND GENERATION

Out of the 6 techniques we employ, the in-context learning experiments (Experiment 1 and Experiment 2) are prompting-based techniques, while Experiments 3 to 6 are fine-tuning-based techniques.

For the prompting techniques, we utilize an 8-shot in-context learning (Figure 10). Since our dataset contains a very wide distribution of questions, the in-context learning prompts are primarily designed to guide the formatting of the different kinds of questions. The prompts are adapted from problems within each of the 8 grades and include different question types and topics. The examples are chosen to avoid biases toward any specific problem type and function as templates for generating solutions, instead of a means for the models to learn numerical reasoning through examples.

Due to resource constraints, a selection of 10 models (Table 7) are used to perform the experiments involving fine-tuning. We use the tokenization from the respective tokenizers that the models were originally trained on. For models with more than 3B parameters, we use low-rank adaptation (LoRA) (Hu et al., 2021) to reduce the number of parameters that need to be trained. In addition, we utilize the 8-bit Adam optimizer (Dettmers et al., 2022) to further reduce the GPU memory usage while training. This allows the respective models to still be trained within the GPU resource constraint. The text is split into blocks of 128 tokens for fine-tuning.

To assist with extraction of the results, we utilize specific tagging to identify the respective questions, explanations and answers. We use the tagging of [QUESTION], [EXPLANATION] and [ANSWER] to precede the respective parts of the problem. This allows us to more easily identify the model’s answer after generation (more details in Subsection B.4).

For model answer generation, unless otherwise specified, we use the default Hugging Face generation parameters. For greedy decoding, we generate responses with a temperature of 0. This ensures that the responses are deterministic. For the self-consistency experiments, the responses are generated with a temperature of 1. This allows a more varied set of reasoning paths to be generated. In our experiments, we perform inference 5 times on each model, allowing up to 5 different answers. The most frequent solution is then chosen to be the self-consistency answer. In the event of a tie, one of the most frequent solutions are chosen at random.

User	You are a helpful AI assistant, tasked with answering math questions. Provide a step-by-step explanation after [EXPLANATION], and your final answer after [ANSWER].
User	[QUESTION] $((5)/(9)) / ((8)/(5)) = ?$
Assistant	[EXPLANATION] To divide fractions, we can multiply the first fraction by the reciprocal of the second fraction. So, $(5/9)$ divided by $(8/5)$ becomes $(5/9)$ multiplied by $(5/8)$, which simplifies to $(25/72)$. [ANSWER] $(25/72)$
User	[QUESTION] There are 3 toy cars, 5 dolls, and 2 teddy bears on display. How many toys are there in total?
Assistant	[EXPLANATION] To find the total number of toys, we add the quantities of each type of toy together: 3 toy cars + 5 dolls + 2 teddy bears = 10 toys in total. [ANSWER] 10
User	[QUESTION] Apply the distributive property to create an equivalent expression. $5(3p - q - 2r) = ?$
Assistant	[EXPLANATION] The distributive property distributes the factor outside the parentheses to each term inside. So, 5 times $3p$ is $15p$, 5 times $-q$ is $-5q$, and 5 times $-2r$ is $-10r$, resulting in the equivalent expression $15p - 5q - 10r$. [ANSWER] $15p - 5q - 10r$
User	[QUESTION] Multiply. $324 * 21$
Assistant	[EXPLANATION] To multiply 324 by 21. Multiply 324 by 1 (the units digit of 21): $324 * 1 = 324$. Multiply 324 by 20 (the tens digit of 21): $324 * 20 = 6480$. Add the results from steps 1 and 2 to get the final product: $324 + 6480 = 6804$. Therefore, 324 multiplied by 21 equals 6804. [ANSWER] 6804
User	[QUESTION] Solve the equation. $6 = (x/4) x = ?$
Assistant	[EXPLANATION] To solve for x , we multiply both sides of the equation by 4 to isolate x . This gives us $x = 6 * 4$, which simplifies to $x = 24$. [ANSWER] 24
User	[QUESTION] Write your answer as a whole number and remainder. $61 / 7 = ?$
Assistant	[EXPLANATION] When dividing 61 by 7, 7 goes into 61 8 times with a remainder of 5. [ANSWER] 8 R 5
User	[QUESTION] The bakery sold $((3)/(5))$ of its cakes in the morning and $((1)/(4))$ of the remaining cakes in the afternoon. How many cakes were sold altogether if there were initially 60 cakes?
Assistant	[EXPLANATION] The bakery initially sold $3/5 * 60 = 36$ cakes in the morning. Then, it sold $1/4 * (60 - 36) = 6$ cakes in the afternoon. Adding these together gives us a total of $36 + 6 = 42$ cakes sold. [ANSWER] 42
User	[QUESTION] David rents a car for a day. The rental company charges an initial fee of 50 dollars plus 20 dollars for each hour the car is rented. If David pays 130 dollars for the rental, write a function C in dollars, where $C(h)$ represents the total cost of renting the car based on the number of hours h it is rented.
Assistant	[EXPLANATION] The total cost C consists of the initial fee of 50 dollars plus 20 dollars for each hour h the car is rented. Thus, the function $C(h) = 50 + 20h$. [ANSWER] $50 + 20h$
User	[QUESTION]

Figure 10: Prompt template for in-context learning to generate solutions. For the experiments without chain-of-thought explanations, the [EXPLANATION] portions are removed. This sequence of prompts are applied using the corresponding chat template.

Table 8: Hyperparameters for fine-tuning and generation of responses

HYPERPARAMETERS	
Fine-tuning	Value
Batch size	16 / 32
Block size	128 tokens
Epochs	10
Early Stopping	2
Optimizer	Adam 8-Bit
Learning rate	5.0×10^{-5}
Weight Decay	0.01
Learning rate schedule	Cosine Annealing
LoRA rank (r)	128
LoRA alpha	64
LoRA dropout	0.05
Generation	Value
Max sample length	250
Sampling Temperature	0 (Greedy decoding) 1 (Self-consistency)

B.4 OUTPUT PARSING AND EVALUATION

To extract the results, we make use of the [QUESTION]-[EXPLANATION]-[ANSWER] format that is introduced in the in-context learning examples and the fine-tuning process. By using this format, we are able to extract any text between ‘[ANSWER]’ and any subsequent ‘[’. The answer is then post-processed to remove any superfluous formatting. To improve the accuracy of answer extraction, we review each model’s outputs and create rule-based processing for each model based on their specific patterns.

After extracting the answers, we then introduce a parsing mechanism to verify if the answer is equivalent to the expected answer. Since our dataset includes answers that involve variables, we use SymPy Meurer et al. (2017) to help simplify the expressions. As part of this process, we convert answers to be parsable by SymPy (e.g. $3y$ is converted to $3 * y$). Then, we apply SymPy’s *sympify* and *simplify* functions to the expected and extracted answers. This ensures that the outputs can be graded fairly and allows us to accurately evaluate cases where answers are mathematically equivalent but expressed differently (e.g. 0.5 is graded equivalently to $1/2$). The performance of the models can then be determined by an equality check between the two simplified SymPy expressions. For answers that cannot be evaluated by SymPy (e.g. equations), we use direct string matching to determine if the extracted answer is accurate. For the experiments, the main evaluation metric employed is the percentage of correctly solved problems (accuracy).

B.5 CALCULATOR ANNOTATIONS

To reduce arithmetic errors within the generation, we utilize calculators in some of the experiments (Experiment 4 and Experiment 5). The aim is to extract expressions that can be calculated using an external solver, accurately calculate them, and then reinsert the evaluated expressions into the model to continue generation. To assist with this, we utilize special tags that identify the start and end of the calculation. We use `||STARTWORK||` to denote the start of an expression, `||ENDWORK||` to denote the end of an expression (without the answer), and `||ENDCALC||` to denote the end of the answer. For instance, $3x + 2x = 5x$ would be expressed as `||STARTWORK|| $3x + 2x$ ||ENDWORK||  $5x$ ||ENDCALC||`.

Using GPT4o, we generate chain-of-thought explanations for each of the problems consisting of these tags. For the purpose of fine-tuning, `||STARTWORK||`, `||ENDWORK||` and `||ENDCALC||` are all added as new individual tokens. When performing inference, models are prompted to generate until an `||ENDWORK||` token, find the preceding `||STARTWORK||` token, and extract the expression in between. This expression is then evaluated using SymPy. We utilize SymPy as our data contains expressions involving variables which cannot be simply evaluated via Python’s `eval` function. After the SymPy evaluation, the answer is appended together with the `||ENDCALC||` token, and the generation continues as per normal (Figure 11).

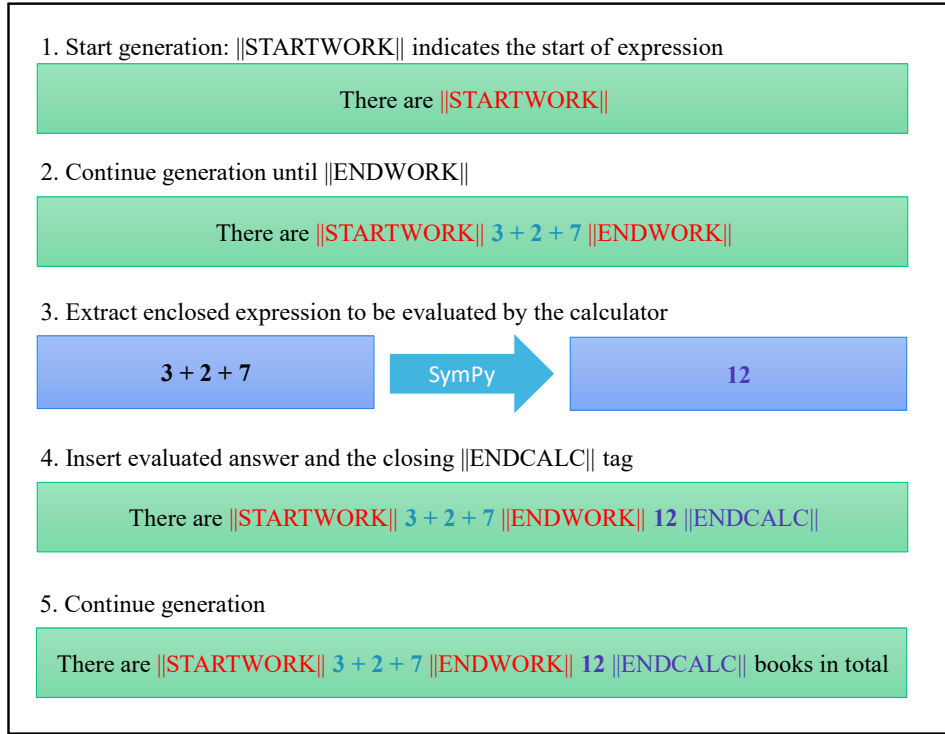


Figure 11: Procedure for model generation with an external solver

We conduct two sets of experiments involving an external calculator. In the initial experiment (Experiment 4), we directly fine-tune the models on the newly generated chain-of-thought explanations with the calculator tags. As a potential improvement to this method, we utilize an external dataset to teach models about the new tokens (Experiment 5). To fully utilize the calculator, it is important that the models can accurately generate the relevant tags so that the correct expressions can be parsed and calculated. Hence, we employ GPT-4o to generate 10000 problems which consists of these tags. We provide seven diverse examples of how the tags should be included within the chain-of-thought explanations, and prompt GPT-4o to generate more problems of a similar nature. In these generated questions, we do not verify the accuracy of the explanations or answers, as the main goal is to have an augmented dataset to familiarize the models with the new tokens used for the calculator. The prompt used to generate the questions is shown in Figure 12.

Below are some questions and sample solutions for Math Word Problems.
There should not be = sign in between the ||STARTWORK|| and ||ENDWORK|| tags. All calculations between ||STARTWORK|| and ||ENDWORK|| tags should be in the format ||STARTWORK|| calculation ||ENDWORK|| result ||ENDCALC||.
Any calculation between the ||STARTWORK|| and ||ENDWORK|| tag must be computable by SymPy and not contain arbitrary words. The result should be a final simplified form of the calculation.

Below are good examples which we aim to generate more of:

[QUESTION] Amy put 9 toppings on her ice cream sundae. She put some cherries and 7 chocolate candies. How many cherries did Amy put on her sundae?

[EXPLANATION] Amy put a total of 9 toppings on her sundae. Since she put 7 chocolate candies, the number of cherries she put can be found by subtracting the number of chocolate candies from the total number of toppings.

||STARTWORK|| 9-7 ||ENDWORK|| 2 ||ENDCALC||

Therefore, Amy put 2 cherries on her sundae.

[ANSWER] 2

[QUESTION] 12 aliens landed in our city. Some aliens were on the first spaceship. The remaining 7 aliens were on the second spaceship. How many aliens were on the first spaceship?

[EXPLANATION] There were 12 aliens in total, and 7 were on the second spaceship. So, the number of aliens on the first spaceship is

||STARTWORK|| 12 - 7 ||ENDWORK|| 5 ||ENDCALC||. Thus, there were 5 aliens on the first spaceship.

[ANSWER] 5

[QUESTION] $((5)/(9)) / ((8)/(5)) = ?$

[EXPLANATION] To divide two fractions, we invert the second fraction (the divisor) and then multiply the two fractions.

So, $(5/9) / (8/5) = (5/9) * (5/8) = < (5/9) * (5/8) = (25/72) > = (25/72)$

[ANSWER] (25/72)

[QUESTION] Apply the distributive property to create an equivalent expression. $(1 - 2g + 4h) * 5 = ?$

[EXPLANATION] To apply the distributive property, you need to multiply each term inside the parentheses by 5.

||STARTWORK|| 1 * 5 ||ENDWORK|| 5 ||ENDCALC||

||STARTWORK|| -2 * 5 ||ENDWORK|| -10 ||ENDCALC||. Hence, we have -10g.

||STARTWORK|| 4 * 5 ||ENDWORK|| 20 ||ENDCALC||. Hence, we have 20h.

Therefore, the equivalent expression is $5 - 10g + 20h$.

[ANSWER] $5 - 10g + 20h$

[QUESTION] A group of friends shared a pizza that cost \$18 equally among 3 people. How much did each person pay?

[EXPLANATION] The total cost of the pizza was \$18, and there were 3 people sharing the pizza. To find out how much each person paid, we divide the total cost by the number of people.

||STARTWORK|| 18 / 3 ||ENDWORK|| 6 ||ENDCALC||

Therefore, each person paid \$6.

[ANSWER] 6

[QUESTION] Multiply. $324 * 21$

[EXPLANATION] To multiply 324 by 21, we multiply each digit of the first number (324) by the second number (21). First, multiply 324 by the units digit of 21, which is 1:

||STARTWORK|| 324 * 1 ||ENDWORK|| 324 ||ENDCALC||

Next, multiply 324 by the tens digit of 21, which is 2, and shift the result one position to the left:

||STARTWORK|| 324 * 20 ||ENDWORK|| 6480 ||ENDCALC||

Finally, add the results from the previous two multiplications:

$324 \text{ (from } 324 * 1) + 6480 \text{ (from } 324 * 20) =$ ||STARTWORK|| 324 + 6480 ||ENDWORK|| 6804 ||ENDCALC||

Therefore, the final result of $324 * 21$ is 6804.

[ANSWER] 6804

[QUESTION] Multiply. $1((1)/(3)) * ((6)/(7))$

[EXPLANATION] To multiply $1((1)/(3))$ by $((6)/(7))$, we first convert the mixed number $1((1)/(3))$ to an improper fraction:

$1((1)/(3)) = 1 + ((1)/(3)) = ((3)/(3)) + ((1)/(3)) = < ((3)/(3)) + ((1)/(3)) = ((4)/(3)) > = ((4)/(3))$

Now, we multiply $((4)/(3))$ by $((6)/(7))$:

$((4)/(3)) * ((6)/(7)) = ((4*6)/(3*7)) =$ ||STARTWORK|| (4*6)/(3*7) ||ENDWORK|| ((24)/(21)) ||ENDCALC||

We can simplify the fraction $((24)/(21))$ by dividing both the numerator and the denominator by their greatest common divisor, which is 3:

$((24)/(21)) =$ ||STARTWORK|| (24/3)/(21/3) ||ENDWORK|| ((8)/(7)) ||ENDCALC||

Therefore, $1((1)/(3)) * ((6)/(7)) = ((8)/(7))$

[ANSWER] $((8)/(7))$

You are forbidden from having more than one ||ENDWORK|| tag within the same calculation.

You are forbidden from equal signs within the ||STARTWORK|| and ||ENDWORK|| tags.

You are forbidden from having calculations between the ||STARTWORK|| and ||ENDWORK|| tags that cannot be processed by SymPy. This means you are forbidden from having variables or arbitrary words in the calculations.

Generate 50 more Math problems that are in the format of the good examples, where there is a question [QUESTION], an explanation

[EXPLANATION] which contains calculations wrapped with the "||STARTWORK|| working ||ENDWORK|| result ||ENDCALC||" tags, followed by an answer [ANSWER]. Include a variety of types of questions.

Do not include any LaTeX or Markdown formatting in the question, explanation, or answer.

You must not violate the forbidden rules mentioned above. NO EXCEPTIONS. There must NOT be any variables in between the tags.

Figure 12: Prompt template for generating more solutions with the appropriate calculator tags. These are generated by GPT-4o in batches of 50 problems, for a total of 10000 problems.

C EXTENDED EXPERIMENT RESULTS

We conduct all experiments 12 times, employing the 6 techniques (Section B.2), each generated with greedy decoding (GD) and self-consistency (SC). The full table of results can be found in Table 9. There are a few outlier situations that are noteworthy. Firstly, the DeepSeekMath model performs better while using in-context learning without chain-of-thought reasoning (ICL without CoT; Experiment 1) provided in the prompts. Upon investigation, this is attributed to the model’s rigid training structure that results in its inflexibility in accepting alternative prompts. Even without prompting the model to think step-by-step, the DeepSeekMath model provides a long explanation before providing the answer. Hence, the additional chain-of-thought prompts we provide may be a hindrance to its generation. Secondly, the Vicuna model is unable to generate any meaningful outputs when in-context learning is utilized without chain-of-thought reasoning (Experiment 1). Especially in the case of greedy decoding generation, the model tends to output the same number repeatedly till the end of generation. Lastly, when the Mistral model is pre-trained with the external dataset for the external calculator (calculator with pre-training; Experiment 5), it has trouble adapting to the new tokenization and starts to instead produce tokens that represent other languages or arbitrary symbols. While the table serves as an overview of the experiment results, the KMX dataset allows us to conduct a progressive analysis of model performance.

Table 9: Accuracy of each model while employing the respective prompting/fine-tuning techniques
GD: greedy decoding generation; **SC**: generation with self-consistency
MEAN*/VARIANCE*: Mean/Variance of the 10 models also used in fine-tuning
MEAN/VARIANCE : Mean/Variance of all 17 models

	ICL without CoT (Expt 1)		ICL with CoT (Expt 2)		Fine-tuning (Expt 3)		Calculator (Expt 4)		Calculator with pre-training (Expt 5)		Identifying Ex. Name (Expt 6)	
	GD	SC	GD	SC	GD	SC	GD	SC	GD	SC	GD	SC
Falcon	0.207	0.184	0.122	0.116	0.314	0.236	0.312	0.266	0.140	0.249	0.320	0.341
Gemma	0.336	0.324	0.359	0.315	0.605	0.524	0.659	0.627	0.672	0.687	0.569	0.592
Llama 3	0.661	0.662	0.755	0.790	0.768	0.813	0.715	0.776	0.751	0.791	0.736	0.758
Mistral	0.465	0.461	0.445	0.447	0.606	0.640	0.589	0.680	0.270	0.429	0.635	0.664
Phi-2	0.538	0.523	0.563	0.589	0.769	0.744	0.738	0.740	0.744	0.688	0.759	0.773
Qwen	0.753	0.747	0.898	0.918	0.900	0.920	0.907	0.922	0.887	0.918	0.907	0.932
TinyLlama	0.187	0.161	0.113	0.104	0.198	0.165	0.382	0.340	0.406	0.390	0.221	0.198
WizardMath	0.583	0.692	0.616	0.722	0.782	0.791	0.800	0.809	0.816	0.814	0.780	0.833
Vicuna	0.134	0.225	0.382	0.440	0.466	0.494	0.577	0.631	0.595	0.676	0.425	0.494
Zephyr	0.441	0.440	0.547	0.602	0.625	0.607	0.646	0.662	0.639	0.646	0.599	0.629
DeepSeekMath	0.906	0.940	0.888	0.936	-	-	-	-	-	-	-	-
Mathstral	0.606	0.622	0.772	0.813	-	-	-	-	-	-	-	-
MetaMath-Mistral	0.583	0.673	0.642	0.720	-	-	-	-	-	-	-	-
Rho	0.335	0.334	0.483	0.416	-	-	-	-	-	-	-	-
Llama 3.1	0.730	0.764	0.858	0.892	-	-	-	-	-	-	-	-
GPT3.5	0.790	0.780	0.870	0.902	-	-	-	-	-	-	-	-
GPT4o-mini	0.793	0.783	0.956	0.948	-	-	-	-	-	-	-	-
MEAN*	0.431	0.442	0.480	0.504	0.603	0.593	0.632	0.645	0.592	0.629	0.595	0.621
VARIANCE*	0.044	0.046	0.063	0.075	0.049	0.061	0.033	0.041	0.060	0.044	0.047	0.051
MEAN	0.532	0.548	0.604	0.628	-	-	-	-	-	-	-	-
VARIANCE	0.054	0.057	0.069	0.078	-	-	-	-	-	-	-	-

Table 10: Accuracy of models using ICL with CoT (Experiment 2), sorted by decreasing accuracy

Model	Accuracy
GPT4o-mini	0.956
Qwen	0.898
DeepSeekMath	0.888
GPT3.5	0.870
Llama 3.1	0.858
Mathstral	0.772
Llama 3	0.755
MetaMath-Mistral	0.642
WizardMath	0.616
Phi-2	0.563
Zephyr	0.547
Rho	0.483
Mistral	0.445
Vicuna	0.382
Gemma	0.359
Falcon	0.122
TinyLlama	0.113

C.1 MODEL PERFORMANCES

To assist with future analysis, we aim to group models by their performance. This categorization helps identify insights amongst models which we demonstrate in future sections.

As an initial check, using ICL with CoT (Experiment 2), we order all the models based on their accuracy (Table 10). However, some of the models have a similar level of accuracy, hence making it difficult to determine a clear categorization solely based on accuracy.

To formally evaluate whether two models are statistically different in terms of their performance, we apply a pairwise t-test. To obtain values for use within the t-test, we use the performance of the models on each topic. The t-test helps determine if the difference in accuracy between two models across various topics is statistically significant or merely due to random fluctuations in the data. The null hypothesis (H_0) for these experiments assume that there are no significant difference between the two models’ accuracies. Hence, if the p-value is below 0.05, we reject the null hypothesis and determine that one model is significantly different from another model across the various topics. This would allow us to establish a clear ordering between the two models.

By repeating this process between all pairs of models, we can organize them into clusters. In essence, models that show no significant difference between each other are grouped into the same cluster, and models that outperform others are grouped into higher or lower performance levels.

As seen from Table 11, this method allows us to categorize the models into 6 approximate clusters. For example, when performing a pairwise t-test against all models, GPT4o-mini outperforms all of them at a 5% significance level. On the other hand, Llama 3.1, GPT3.5-turbo, Qwen and DeepSeek-Math show no statistical significance between each other, and thus can be grouped together as a cluster.

This statistical approach prevents arbitrary distinctions between models that perform similarly, leading to a more principled ordering of the models. We also utilize this form of statistical tests for other experiments when comparing models.

Table 11: p-value for pairwise t-test between models based on accuracy of each topic. Red cells denote a p-value of over 0.05. This creates a categorization of models as indicated by the colouring on the first column.

	TinyLlama	Falcon	Gemma	Vicuna	Mistral	Rho	Zephyr	Phi-2	WizardMath	MetaMath-Mistral	Mathstral	Llama 3	Llama 3.1	GPT3.5	Owen	DeepSeek Math	GPT4o-mini
TinyLlama		0.1555	0.0004	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Falcon	0.1555		0.0013	0.0004	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Gemma	0.0004	0.0013		0.4623	0.0088	0.0191	0.0009	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Vicuna	0.0000	0.0004	0.4623		0.0381	0.0642	0.0015	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Mistral	0.0000	0.0000	0.0088	0.0381		0.9527	0.0121	0.0063	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Rho	0.0000	0.0000	0.0191	0.0642	0.9527		0.0474	0.0036	0.0004	0.0001	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Zephyr	0.0000	0.0000	0.0009	0.0015	0.0121	0.0474		0.6027	0.0001	0.0034	0.0001	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Phi-2	0.0000	0.0000	0.0000	0.0000	0.0063	0.0036	0.6027		0.1308	0.0330	0.0002	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
WizardMath	0.0000	0.0000	0.0000	0.0000	0.0000	0.0004	0.0001	0.1308		0.3123	0.0015	0.0001	0.0000	0.0000	0.0000	0.0000	0.0000
MetaMath-Mistral	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0034	0.0330	0.3123		0.0199	0.0006	0.0000	0.0000	0.0000	0.0000	0.0000
Mathstral	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0002	0.0015	0.0199		0.7223	0.0016	0.0021	0.0024	0.0003	0.0000
Llama 3	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0006	0.7223		0.0016	0.0004	0.0024	0.0002	0.0000
Llama 3.1	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0011	0.0016		0.8263	0.3905	0.1126	0.0000
GPT3.5	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0021	0.0004	0.8263		0.3785	0.1116	0.0005
Owen	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0003	0.0002	0.3905	0.3785		0.5638	0.0080
DeepSeekMath	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0003	0.0002	0.1126	0.1116	0.5638		0.0016
GPT4o-mini	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0005	0.0080	0.0016	

C.2 ERROR DETECTION

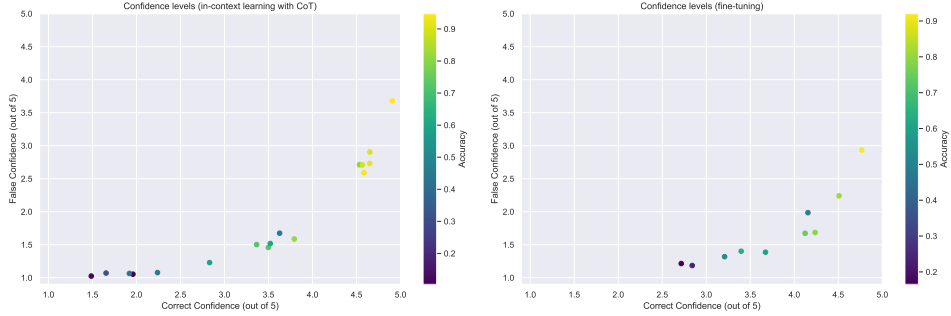


Figure 13: Confidence levels of models. Relationship between correct confidence and false confidence with the following methods **Left**: ICL with CoT (Experiment 2); **Right**: fine-tuning (Experiment 3)

Calibration has been identified as an issue for modern neural networks (Guo et al., 2017). We explore ways to quantify the uncertainty of a model and discuss if there are similar calibration issues within language models. To achieve this, we utilize the results generated from the experiments involving self-consistency. This involves generating 5 completions from each model using a high temperature generation setting. The outputs are then parsed to retrieve an answer for each completion, and a majority vote is performed to determine the final answer presented by the model.

As a proxy for uncertainty estimation, we calculate 4 metrics. An example of how these metrics work is provided in Figure 14.

- **Correct Confidence** measures the average number of results that contribute to the majority vote when the majority answer is correct. With a high Correct Confidence, it suggests that when the model gets the right answer, it does so with strong consensus across individual results. A model that is well-calibrated should have a high Correct Confidence because it signals that the model is correctly detecting cases where it can confidently predict the right outcome. This metric ranges from 1 to 5, where 1 indicates that only one of the generated answers are correct when a correct answer is given, and 5 indicates that all generated answers are in agreement whenever the final answer is correct. Note that this does not imply 100% accuracy, as questions that are answered incorrect are not considered in this metric.
- **False Confidence** measures the average number of results that contribute to the majority vote when the majority answer is incorrect. A high False Confidence indicates that the model is frequently confident when it is wrong, suggesting overconfidence in certain situations. This can help us understand overconfident failures, which is important for model robustness. False Confidence also ranges from 1 to 5, where 1 indicates that only one of the generated answers provide the specific wrong answer when that answer is chosen as the model’s selected answer.
- **Answer Consistency** measures the average number of results that contribute to the majority vote, regardless of whether that answer is correct or incorrect. This forms a baseline on how confident the model is in general, and is useful to look at in relation to Correct Confidence and False Confidence. Similar to Correct Confidence and False Confidence, Answer Consistency can start from a minimum of 1, to a maximum of 5.
- **Hidden Correctness** measures the average number of correct results when the majority vote is incorrect. High Hidden Correctness indicates that even when the model’s majority vote is wrong, some results generated are still correct. This suggests potential in examining the answers produced and improving how we determine answers or detect errors. Since the majority vote is incorrect, this signifies that the number of correct answers has to be less than the number of incorrect answers. Hence, the hidden correctness ranges from 0 to 2, where 2 would indicate that whenever a model is incorrect, it produces 2 answers that are correct, but also 2 or 3 of the same wrong answer which gets selected as the model’s answer instead.

Generated Answers: 11, 11, 11, 12, 12 Model Output: 11 (since it is the majority)	Generated Answers: 11, 12, 13, 14, 15 Model Output: 11 (ties broken arbitrarily)
Case 1: Actual Answer is 11 Correct Confidence: 3 False Confidence: - Answer Consistency: 3 Hidden Correctness: -	Case 1: Actual Answer is 11 Correct Confidence: 1 False Confidence: - Answer Consistency: 1 Hidden Correctness: -
Case 2: Actual Answer is 12 Correct Confidence: - False Confidence: 3 Answer Consistency: 3 Hidden Correctness: 2	Case 2: Actual Answer is 12 Correct Confidence: - False Confidence: 1 Answer Consistency: 1 Hidden Correctness: 1
Case 3: Actual Answer is 13 Correct Confidence: - False Confidence: 3 Answer Consistency: 3 Hidden Correctness: 0	Case 3: Actual Answer is 16 Correct Confidence: - False Confidence: 1 Answer Consistency: 1 Hidden Correctness: 0

Figure 14: Example of metrics for uncertainty estimation. A '-' denotes that the metric is not relevant in that situation and hence does not contribute to the final calculation of the metric.

We discover a high correlation between a model’s Correct Confidence and its accuracy. For ICL with CoT (Experiment 2), GPT4o-mini exhibits the highest Correct Confidence (4.91 of the 5 results provide the correct answer when the majority vote is correct), suggesting that it has the highest degree of agreement when it makes correct predictions. This is aligned with its high accuracy of 94.76%. GPT-3.5 and Qwen show slightly lower Correct Confidence values (4.65 each) but still perform well overall (with accuracies of 90.16% and 91.83% respectively). On the other hand, models like TinyLlama, Rho, Gemma and Falcon have a Correct Confidence value of below 2. This indicates that their predictions tend to be more uncertain or unstable even when correct, and correlates with their lower overall accuracy. This correlation is a useful property for models as it shows that accurate models also are confident about the correct answers they provide.

However, we observe that the highly accurate models also have a high False Confidence score. GPT4o-mini and DeepSeekMath have high False Confidence values (3.68 and 2.59 respectively), suggesting that when they are wrong, they still have strong agreement in their incorrect answers. This indicates possible overconfidence in the models which is detrimental in important applications. Some models such as Mistral or Gemma have a lower False Confidence value, indicating less overconfidence. However, these models display low accuracy and have low Answer Consistency values (1.60 and 1.25 respectively) in general, suggesting that they lack confidence in general, rather than providing accurate uncertainty estimations in incorrect situations.

GPT4o-mini and DeepSeekMath have the highest Hidden Correctness values (0.82 and 0.77 respectively), indicating that even when these models produce incorrect majority answers, there is a decent chance that there exist a correct answer generated. This opens the possibility of researching how to better take advantage of these hidden correct answers.

In general, we observe a high direct relationship between the confidence of a model when it produces a correct answer and when it produces an incorrect answer. In the GPT4o-mini model, the model exhibits a high Correct Confidence score (4.91 of the 5 results provide the correct answer when the majority vote is correct), but a high False Confidence score (3.68 of the 5 results contribute to the same wrong answer when an incorrect answer is provided). In contrast, the less accurate models show lower confidence overall, both in correct and incorrect answer.

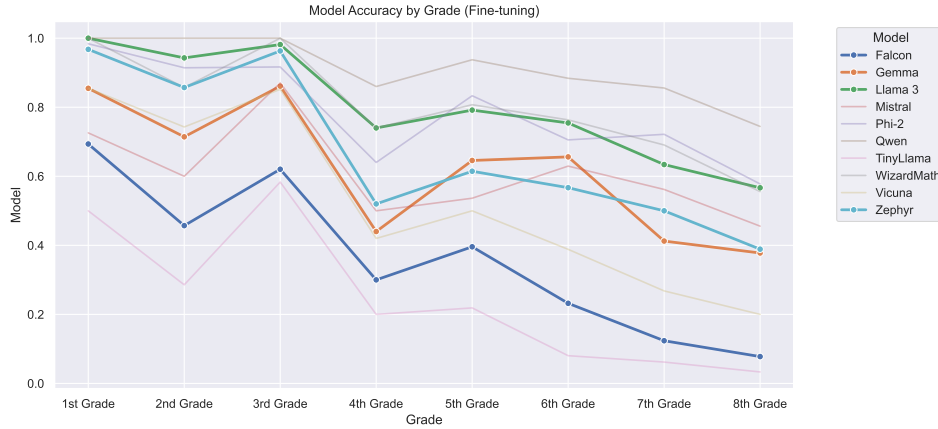


Figure 15: Model accuracy across different grades after fine-tuning (Experiment 3)

Future research can delve deeper into understanding whether reducing model overconfidence in incorrect answers could help strike a better balance between accuracy and reliability, providing more nuanced decision-making processes when uncertainty arises in language models. Additionally, studying how models could incorporate more nuanced decision-making processes when uncertainty arises could lead to improved mathematical reasoning and error detection. The KMX dataset offers an opportunity to explore these nuances further and to investigate how these confidence dynamics manifest across different models.

C.3 GRADE

As noted earlier, there is a general decrease in accuracy across the grades. This is aligned with our expectation that the more advanced grades contain questions that are more challenging and may require more reasoning steps. However, this decrease in accuracy is not consistent. We show that while using in-context learning with chain-of-thought reasoning, there is an unexpected dip in performance in the 2nd and 4th grades. While conducting other experiments with other techniques, we observe that this trend is evident as well (Table 14).

To verify that the trend is similar across all the models, we perform a pairwise Spearman correlation test using the accuracy values for each model across the 8 grades. This statistical test helps to measure how well two variables can be described using a monotonic function, which in this case refers to whether the patterns of accuracy across grades follow a similar order between models. A correlation of 1 indicates a perfect agreement between the model and all other models, a correlation of 0 indicates no correlation, and a correlation of -1 indicates a perfect disagreement. An example of how this test is applied can be seen in Table 12, where we perform the statistical tests based on the results of using ICL with CoT (Experiment 2) applied. We notice that Falcon and TinyLlama both have lower correlation values. This is due to both these models performing consistently low across the grades, and hence the pattern signal is not as obvious. Hence, we also compute an aggregation of the average Spearman correlation excluding these two models. The mean correlation for the models range between 0.81 to 0.91, which signifies a very strong correlation. Hence, the unstable ordering of the grades can be shown to be verifiably similar across all the models.

While performing similar analysis on the different techniques applied (Table 14), we observe that this trend remains prevalent. In fact, this trend is even more prominent in the techniques which involve fine-tuning the model. We visually observe the same dips at the 2nd and 4th grade (Figure 15) in Experiment 3 and perform similar statistical tests (Table 13) that show the strong correlation between the trends in each of the models.

This suggests that grade level alone is not always a reliable indicator of a problem’s difficulty for language models; humans and models have differences in what they deem as challenging. These dips indicate that models may struggle with particular types of questions, skills, or concepts that are more prevalent in these grades, and even fine-tuning on similar questions is insufficient to close these gaps

Table 12: Spearman correlation coefficient for each model against the other models for ICL with CoT (Experiment 2). This shows the similarity of trends in grades between models. The second last column denotes the mean correlation of the model, while the last column denotes the mean correlation excluding the correlation with Falcon and TinyLlama (which have low accuracies throughout all grades and hence do not display a clear trend)

	Falcon	Gemma	Llama 3	Mistral	Phi-2	Qwen	TinyLlama	Wizard Math	Vicuna	Zephyr	DeepSeek Math	Mathstral	MetaMath-Mistral	Rho	Llama 3.1	GPT3.5	GPT4o-mini	Mean (excl Falcon, TinyLlama)
Falcon	None	0.548	0.524	0.333	0.524	0.395	0.619	0.452	0.619	0.548	0.571	0.381	0.238	0.548	0.690	0.476	0.395	0.491
Gemma	0.548	None	0.833	0.714	0.905	0.922	0.786	0.881	0.952	0.952	0.833	0.857	0.833	0.952	0.929	0.929	0.850	0.855
Llama 3	0.524	0.833	None	0.762	0.857	0.862	0.643	0.905	0.833	0.833	0.929	0.738	0.881	0.929	0.881	0.881	0.695	0.812
Mistral	0.333	0.714	0.762	None	0.905	0.850	0.548	0.929	0.833	0.810	0.786	0.786	0.833	0.667	0.762	0.833	0.826	0.761
Phi-2	0.524	0.905	0.857	0.905	None	0.970	0.690	0.976	0.976	0.905	0.929	0.833	0.881	0.857	0.952	0.881	0.898	0.871
Qwen	0.395	0.922	0.862	0.850	0.970	None	0.731	0.970	0.934	0.874	0.898	0.886	0.922	0.898	0.910	0.874	0.928	0.864
TinyLlama	0.619	0.786	0.643	0.548	0.890	0.731	None	0.714	0.714	0.690	0.571	0.905	0.571	0.786	0.690	0.714	0.850	0.701
WizardMath	0.452	0.881	0.905	0.929	0.976	0.970	0.714	None	0.929	0.881	0.905	0.881	0.905	0.881	0.905	0.905	0.898	0.870
Vicuna	0.619	0.952	0.833	0.833	0.976	0.934	0.714	0.929	None	0.952	0.905	0.810	0.833	0.881	0.976	0.905	0.862	0.870
Zephyr	0.548	0.952	0.833	0.810	0.905	0.874	0.690	0.881	0.952	None	0.833	0.810	0.857	0.881	0.905	0.976	0.802	0.844
DeepSeekMath	0.571	0.833	0.929	0.786	0.929	0.898	0.690	0.905	0.905	None	None	0.667	0.881	0.857	0.952	0.810	0.731	0.816
Mathstral	0.381	0.857	0.738	0.786	0.833	0.886	0.905	0.881	0.810	0.810	0.667	None	0.786	0.833	0.738	0.857	0.958	0.817
MetaMath-Mistral	0.238	0.833	0.881	0.833	0.881	0.922	0.571	0.905	0.833	0.857	0.881	0.786	None	0.833	0.810	0.881	0.802	0.797
Rho	0.548	0.952	0.929	0.667	0.857	0.898	0.786	0.881	0.881	0.881	0.857	0.833	0.833	None	0.905	0.905	0.778	0.837
Llama 3.1	0.690	0.929	0.881	0.762	0.952	0.910	0.690	0.905	0.976	0.905	0.952	0.738	0.810	0.905	None	0.857	0.790	0.853
GPT3.5	0.476	0.929	0.881	0.833	0.881	0.874	0.714	0.905	0.905	0.976	0.810	0.857	0.881	0.905	0.857	None	0.802	0.843
GPT4o-mini	0.395	0.850	0.695	0.826	0.898	0.928	0.850	0.898	0.862	0.802	0.731	0.958	0.802	0.778	0.790	0.802	None	0.804

Table 13: Mean pairwise Spearman correlation coefficient of each model with all other models. A high correlation indicates a similarity between the two models in the trend observed

Model	Fine-tuning (Expt 3)	Calculator (Expt 4)	ID Exercise (Expt 6)
Falcon	0.930	0.786	0.909
Gemma	0.925	0.735	0.941
Llama 3	0.952	0.856	0.878
Mistral	0.790	0.786	0.871
Phi-2	0.893	0.839	0.907
Qwen	0.936	0.892	0.913
TinyLlama	0.925	0.896	0.938
WizardMath	0.955	0.875	0.925
Vicuna	0.930	0.875	0.941
Zephyr	0.952	0.893	0.946

Table 14: Model accuracy of the math problems across different grades using
Top: ICL without CoT (Experiment 1) **Bottom:** Fine-tuning (Experiment 3)

Model	1st Grade	2nd Grade	3rd Grade	4th Grade	5th Grade	6th Grade	7th Grade	8th Grade
Falcon	0.4194	0.3714	0.6389	0.24	0.1406	0.1205	0.1082	0.0333
Gemma	0.5645	0.4286	0.7593	0.24	0.3438	0.2857	0.1546	0.1889
Llama 3	0.9839	1	0.9537	0.66	0.6719	0.6696	0.4227	0.4222
Mistral	0.8387	0.8286	0.7593	0.46	0.4896	0.4286	0.2216	0.2778
Phi-2	0.9677	0.7714	0.8056	0.46	0.599	0.4911	0.3247	0.3222
Qwen	1	0.9429	0.9537	0.76	0.7969	0.7277	0.5722	0.6222
TinyLlama	0.4194	0.3714	0.5741	0.22	0.1562	0.0848	0.067	0.0556
WizardMath	0.7581	0.8857	0.8611	0.42	0.5885	0.5714	0.4433	0.4222
Vicuna	0.2903	0.3143	0.1759	0.12	0.0833	0.1473	0.0412	0.1889
Zephyr	0.9194	0.6286	0.8241	0.34	0.4635	0.3884	0.2113	0.2111
DeepSeekMath	1	0.9429	0.963	0.76	0.8802	0.942	0.8814	0.8556
Mathstral	0.8548	0.9143	0.8519	0.6	0.5833	0.625	0.3866	0.5
MetaMath-Mistral	0.9355	0.8571	0.7778	0.42	0.625	0.5268	0.4381	0.4556
Rho	0.5484	0.5143	0.7037	0.22	0.3646	0.2946	0.1907	0.0889
Llama 3.1	0.9677	1	0.9722	0.54	0.7656	0.6964	0.5773	0.6111
GPT3.5	1	1	0.9907	0.66	0.8125	0.8036	0.6082	0.7
GPT4o-mini	1	1	1	0.78	0.8073	0.8214	0.6237	0.5889
MEAN	0.792	0.751	0.798	0.465	0.540	0.507	0.369	0.385

Model	1st Grade	2nd Grade	3rd Grade	4th Grade	5th Grade	6th Grade	7th Grade	8th Grade
Falcon	0.6935	0.4571	0.6204	0.3	0.3958	0.2321	0.1237	0.0778
Gemma	0.8548	0.7143	0.8611	0.44	0.6458	0.6562	0.4124	0.3778
Llama 3	1	0.9429	0.9815	0.74	0.7917	0.7545	0.634	0.5667
Mistral	0.7258	0.6	0.8704	0.5	0.5365	0.6295	0.5619	0.4556
Phi-2	0.9839	0.9143	0.9167	0.64	0.8333	0.7054	0.7216	0.5778
Qwen	1	1	1	0.86	0.9375	0.8839	0.8557	0.7444
TinyLlama	0.5	0.2857	0.5833	0.2	0.2188	0.0804	0.0619	0.0333
WizardMath	1	0.8571	1	0.74	0.8073	0.7634	0.6907	0.5556
Vicuna	0.8548	0.7429	0.8519	0.42	0.5	0.3884	0.268	0.2
Zephyr	0.9677	0.8571	0.963	0.52	0.6146	0.567	0.5	0.3889
MEAN	0.858	0.737	0.865	0.536	0.628	0.566	0.483	0.398

in reasoning for the language models. This motivates a deeper investigation into additional problem dimensions which could better help us understand the nuances of a math problem.

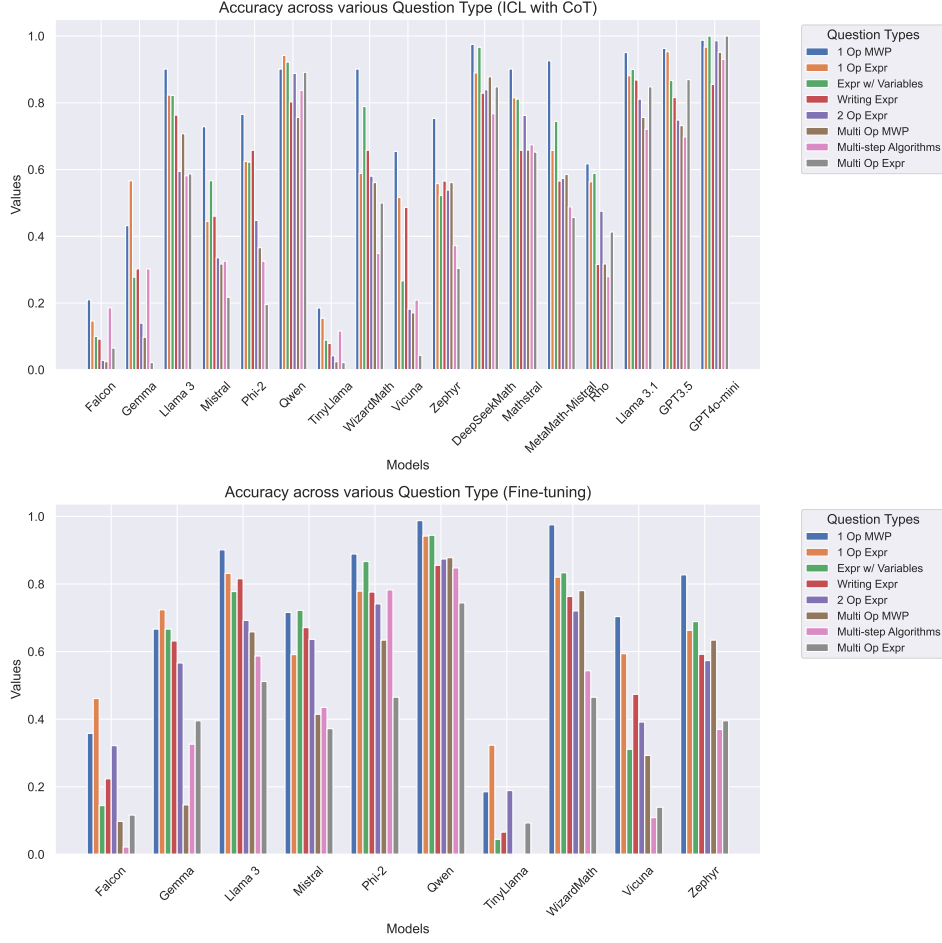


Figure 16: Accuracy of models for each question type using **Top**: ICL with CoT (Experiment 2); **Bottom**: Fine-tuning (Experiment 3)

C.4 QUESTION TYPE

To facilitate a more fine-grained analysis, we further divide ‘Expressions’ and ‘Word Problems’ based on the number of distinct types of mathematical operations involved. For this analysis, we focus on question types that contain at least 40 problems (8 out of the 12 question types).

A full table of results is provided in Table 15, where we evaluate each of the 17 models using ICL with CoT (Experiment 2) on the 8 question types. For each model, we also perform aggregations to obtain the mean, variance and coefficient of variation (CV). The CV is used to measure the relative variation of data around the mean. This is particularly useful in cases where various data sets may contain different means, and thus serves as a way to assess the relative variability in performance between the models. For models with higher mean across all question types, a small deviation may appear large compared to a poor-performing model. Thus, the coefficient of variation seeks to normalize this by computing the standard deviation (σ) as a function of the mean (μ):

$$CV = \frac{\sigma}{\mu} \quad (1)$$

Besides the hierarchy of question types as discussed earlier, we observe a strong correlation between all the question types. A model that performs well relative to other models in one question type is likely to perform in the same way in another question type. We verify this with a pairwise Spearman correlation test between each question type. The 17 model accuracy values are used as the data points and between each question type, the correlation ranges from 0.89 to 0.98, signifying a very

Table 15: Accuracy scores per question type for each model while employing ICL with CoT (Experiment 2)

Model	One OpType MWP	One OpType Expr	Expressions with Variables	Write Expression	Two OpType Expr	Multi OpType MWP	Multi-step Algorithm	Multi OpType Expr	Mean	Variance	Coeff of Variation (CV)
Falcon	0.210	0.146	0.100	0.092	0.028	0.024	0.186	0.065	0.107	0.005	0.649
Gemma	0.432	0.566	0.278	0.303	0.140	0.098	0.302	0.022	0.268	0.032	0.669
Llama 3	0.901	0.823	0.822	0.763	0.594	0.707	0.581	0.587	0.722	0.016	0.172
Mistral	0.728	0.445	0.567	0.461	0.336	0.317	0.326	0.217	0.425	0.027	0.384
Phi-2	0.765	0.624	0.622	0.658	0.448	0.366	0.326	0.196	0.501	0.039	0.392
Qwen	0.901	0.942	0.922	0.803	0.888	0.756	0.837	0.891	0.868	0.004	0.073
TinyLlama	0.185	0.155	0.089	0.079	0.042	0.024	0.116	0.022	0.089	0.004	0.676
WizardMath	0.901	0.588	0.789	0.658	0.580	0.561	0.349	0.500	0.616	0.029	0.277
Vicuna	0.654	0.517	0.267	0.487	0.182	0.171	0.209	0.044	0.316	0.044	0.667
Zephyr	0.753	0.558	0.522	0.566	0.539	0.561	0.372	0.304	0.522	0.018	0.259
DeepSeekMath	0.975	0.890	0.967	0.829	0.839	0.878	0.767	0.848	0.874	0.005	0.080
Mathstral	0.901	0.815	0.811	0.658	0.762	0.659	0.674	0.652	0.742	0.009	0.127
MetaMath-Mistral	0.926	0.658	0.744	0.566	0.573	0.585	0.488	0.457	0.625	0.023	0.243
Rho	0.617	0.564	0.589	0.316	0.476	0.317	0.279	0.413	0.446	0.018	0.302
Llama 3.1	0.951	0.881	0.900	0.868	0.811	0.756	0.721	0.848	0.842	0.006	0.090
GPT3.5	0.963	0.953	0.867	0.816	0.748	0.732	0.698	0.870	0.831	0.010	0.120
GPT4o-mini	0.988	0.967	1.000	0.855	0.986	0.951	0.930	1.000	0.960	0.002	0.051

strong positive correlation between each question type. With the ordering of the performances of models across different question types (Figure 16), this shows that a good performance in multi-step algorithms requires an ability to solve simpler math word problems or expressions. We can demonstrate this hierarchy by observing the variance in the results across the question types. Models such as Qwen and DeepSeekMath perform well consistently, having a variance of 0.004 and 0.005 respectively. On the other hand, models such as Vicuna have a lower mean of 0.316 and a variance of 0.044. The explanation of the variance can be seen from Figure 16 where Vicuna performs reasonably well in math word problems with one type of operation, followed by a sharp drop in accuracy in writing expressions or solving expressions with one type of operation, followed by another drop in accuracy other question types. This ‘difficulty levels’ in question types can be seen in the other models such as Phi-2 or Rho, suggesting some potential prerequisites in solving certain types of questions.

Furthermore, stronger models not only demonstrate good performance in mean accuracy, but are also consistent in their results. From Table 15, we observe that GPT4o-mini has the highest mean of 0.960, and the lowest CV of 0.051. Meanwhile, TinyLlama has a mean of 0.089, and the highest CV of 0.676. As an additional layer of verification, we conduct a Pearson correlation test between the mean of a model and the CV. The resulting Pearson coefficient was -0.959 with a p-value of 1.2×10^{-9} , signifying an almost perfect inverse relation between the mean and the coefficient of variation.

These observations are also applicable to the other techniques that we experimented on (Table 16). Similarly, we see that models that perform well have a low variance, while the weaker models tend to have a higher variance. In weaker models, they demonstrate different performances for various question types which further support the idea of different question types having a difficulty ordering for models. The Pearson coefficient between the mean and the CV are between -0.95 and -0.97, which show that stronger models are more stable, regardless of technique employed.

In conclusion, our analysis reveals an underlying structure behind math problems. We observe that simpler question types, such as math word problems with one type of operation, generally outperform more complex tasks, such as multi-step algorithms and multi-operation expressions. This is consistent across most models and highlights how mathematical reasoning capabilities develop as the complexity of the problems increases. The high correlations between different question types suggest that excelling in simpler questions forms a foundation for tackling more complex ones, prompting possible research into building these layers within models for mathematical reasoning. The KMX dataset enables this fine-grained analysis, providing diversity in question types to assess a model’s ability to generalize across mathematical problems. To further unravel how models perform in different mathematical domains, we explore how a topic-based analysis can give us insights on evaluating the dimensions of mathematical reasoning and the robustness of models in dealing with this diversity.

Table 16: Accuracy scores per question type for each model while employing **Top**: ICL without CoT (Experiment 1); **Bottom**: Fine-tuning (Experiment 3)

Model	One OptType MWP	One OptType Expr	Expressions with Variables	Write Expression	Two OptType Expr	Multi OptType MWP	Multi-step Algorithm	Multi OptType Expr	Mean	Variance	Coeff of Variation (CV)
Falcon	0.235	0.365	0.171	0.011	0.209	0.007	0.000	0.000	0.125	0.020	1.121
Gemma	0.395	0.561	0.303	0.267	0.279	0.049	0.073	0.000	0.241	0.036	0.793
Llama 3	0.889	0.848	0.697	0.567	0.465	0.385	0.366	0.500	0.590	0.041	0.342
Mistral	0.778	0.624	0.540	0.444	0.349	0.168	0.049	0.000	0.369	0.078	0.758
Phi-2	0.765	0.710	0.605	0.567	0.302	0.308	0.146	0.109	0.439	0.065	0.580
Qwen	0.901	0.923	0.855	0.689	0.581	0.601	0.244	0.326	0.640	0.065	0.399
TinyLlama	0.210	0.318	0.118	0.022	0.116	0.077	0.024	0.022	0.113	0.011	0.925
WizardMath	0.864	0.577	0.618	0.711	0.395	0.504	0.585	0.500	0.594	0.021	0.241
Vicuna	0.346	0.113	0.092	0.244	0.116	0.042	0.098	0.000	0.131	0.013	0.849
Zephyr	0.654	0.655	0.474	0.300	0.326	0.140	0.098	0.044	0.336	0.058	0.715
DeepSeekMath	0.975	0.901	0.895	0.978	0.767	0.881	0.927	0.891	0.902	0.004	0.073
Mathstral	0.852	0.749	0.645	0.689	0.512	0.350	0.220	0.152	0.521	0.066	0.493
MetaMath-Mistral	0.938	0.635	0.382	0.600	0.512	0.476	0.537	0.326	0.551	0.035	0.341
Rho	0.296	0.572	0.290	0.144	0.140	0.161	0.171	0.109	0.235	0.023	0.649
Llama 3.1	0.864	0.876	0.803	0.633	0.651	0.587	0.244	0.587	0.656	0.042	0.312
GPT3.5	0.951	0.939	0.829	0.711	0.628	0.629	0.537	0.348	0.696	0.043	0.296
GPT4o-mini	0.963	0.948	0.882	0.756	0.605	0.608	0.610	0.261	0.704	0.055	0.333

Model	One OptType MWP	One OptType Expr	Expressions with Variables	Write Expression	Two OptType Expr	Multi OptType MWP	Multi-step Algorithm	Multi OptType Expr	Mean	Variance	Coeff of Variation (CV)
Falcon	0.358	0.461	0.144	0.224	0.322	0.098	0.022	0.116	0.218	0.023	0.690
Gemma	0.667	0.724	0.667	0.632	0.566	0.146	0.326	0.395	0.515	0.042	0.396
Llama 3	0.901	0.832	0.778	0.816	0.692	0.659	0.587	0.512	0.722	0.018	0.184
Mistral	0.716	0.591	0.722	0.671	0.636	0.415	0.435	0.372	0.570	0.020	0.249
Phi-2	0.889	0.779	0.867	0.776	0.741	0.634	0.783	0.465	0.742	0.019	0.184
Qwen	0.988	0.942	0.944	0.855	0.874	0.878	0.848	0.744	0.884	0.006	0.085
TinyLlama	0.185	0.323	0.044	0.066	0.189	0.000	0.000	0.093	0.113	0.013	0.996
WizardMath	0.975	0.820	0.833	0.763	0.720	0.781	0.544	0.465	0.738	0.027	0.222
Vicuna	0.704	0.594	0.311	0.474	0.392	0.293	0.109	0.140	0.377	0.043	0.552
Zephyr	0.827	0.663	0.689	0.592	0.573	0.634	0.370	0.395	0.593	0.023	0.255

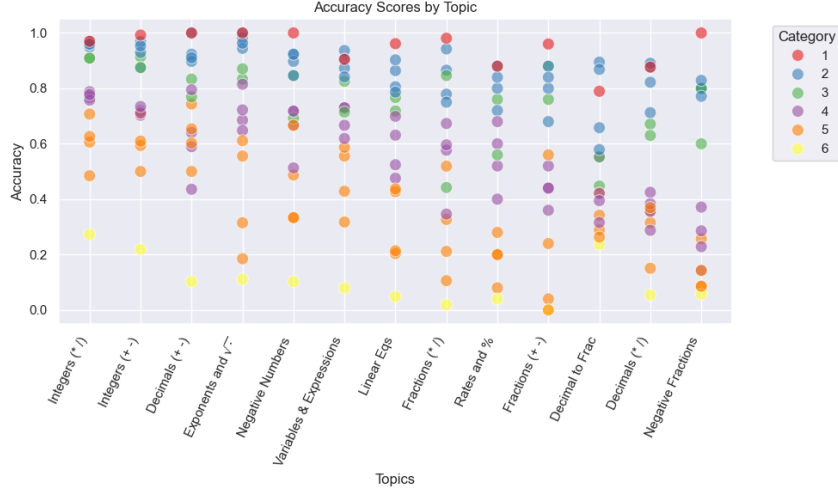


Figure 17: Model performances across topics. The models are grouped into 6 categories based on the ordering determined in subsection C.1, where category 1 consists of the best models, and category 6 consists of the worst-performing models

C.5 TOPIC

To ensure sufficient data for analysis, we analyze topics that contain at least 25 problems (13 out of the 18 topics). Similar to the analysis conducted in Subsection C.4, we calculate the mean, variance and coefficient of variation (CV) for each model across the different topics. We note that the inverse relationship between the mean and the CV still holds (for Experiment 2, ICL with CoT, the Pearson correlation is -0.944 with a p-value of 1.3×10^{-8}), implying that the stronger topics have a higher consistency across the topics. As an additional analysis, we observe the performances of the models on each topic. Based on the ordering of the models as determined in Subsection C.1, we colour-code each model into their respective category and plot their results. For each model that is in a ‘better-performing category’, they perform better than almost all models in a lower category for every topic. This shows a categorization of how models of different capabilities can progressively perform, and provides insights as to the aspects that can be focused on to improve a model’s numerical reasoning abilities.

Besides performing analysis at a model-level, we also inspect another dimension and focus on how models perform on each topic and the relationship between each topic. In this analysis (Figure 18), we discuss three of the techniques used for our experiments: ICL without CoT reasoning (Experiment 1), ICL with CoT reasoning (Experiment 2) and fine-tuning (Experiment 3). During our aggregations of the results, we exclude Vicuna from the ICL without CoT analysis and Falcon from the ICL with CoT reasoning analysis. The rationale for this is that these models struggle to adapt to the formats we provide, producing nonsensical outputs a majority of the time. For instance, Falcon would continually repeat the in-context learning prompts instead of generating an answer that is related to the problem. In contrast, although TinyLlama performs poorly, it still produces reasoning that is relevant to the problem. Therefore, to provide a more accurate analysis of how models reason about individual topics, we exclude the Falcon and Vicuna models in the respective analyses, as their results are not reflective of the difficulty of the topic for mathematical reasoning.

In all the techniques, we observe that the problems involving integers have a high performance and very low variance across all the models. For instance, while employing ICL with CoT (Experiment 2), integer problems involving addition and subtraction have a mean of 0.751 and variance of 0.042 , while integer problems involving multiplication and division have a mean of 0.776 and a variance of 0.040 across the models. In the case of ICL with CoT (Experiment 2) and fine-tuning (Experiment 3), we observe that as the models face more challenging topics, the mean performance decreases but the variance increases. This suggests that the better models are still able to perform reasonably well on these topics that other models find difficult. Problems involving negative fractions is considered one of the most difficult topics by the models. In this case, the variance of the

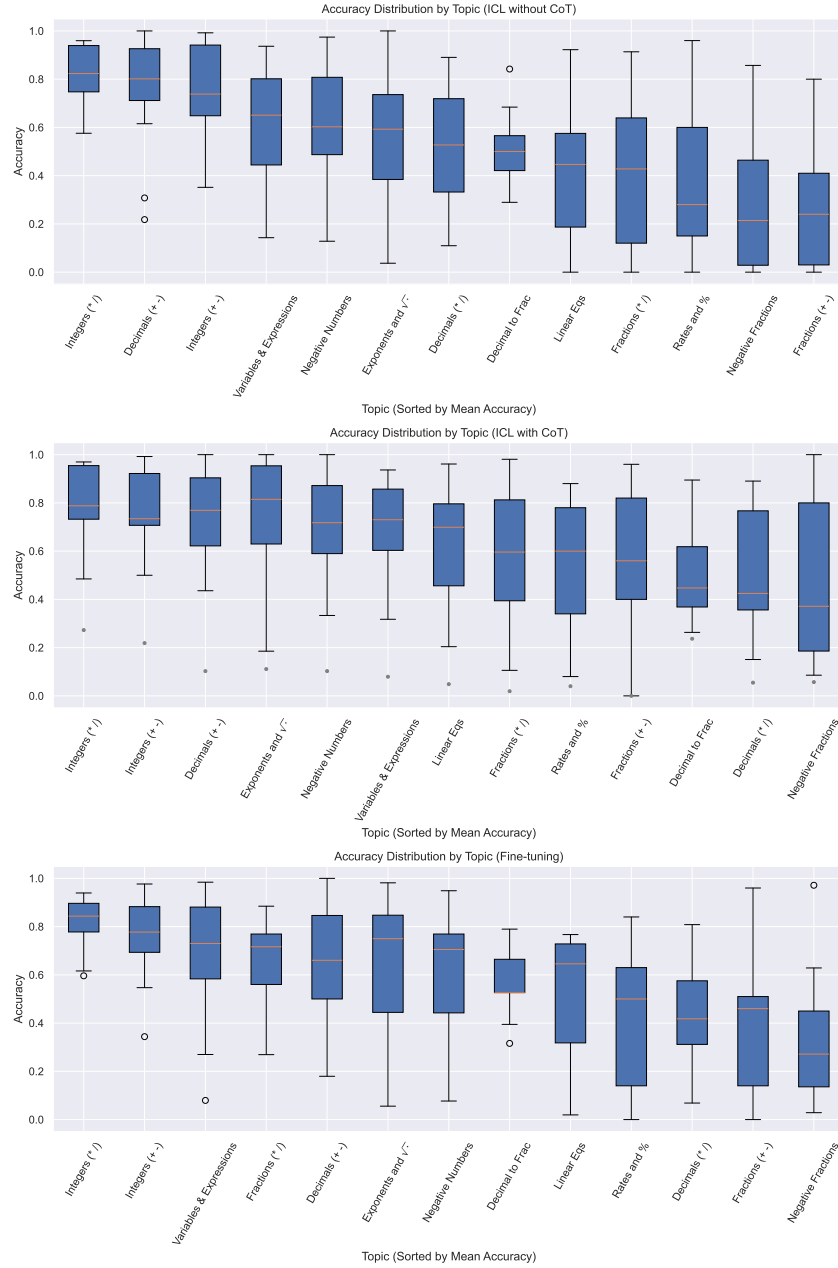


Figure 18: Model accuracy across topics for **Top:** ICL without CoT (Experiment 1); **Middle:** ICL with CoT (Experiment 2); **Bottom:** Fine-tuning (Experiment 3)
For ICL with CoT, TinyLlama is denoted as a grey circle due to its general inability to answer questions

models for ICL with CoT increases to 0.111, with a mean of 0.454. GPT4o-mini is able to get every problem correct, while DeepSeekMath, Qwen and Mathstral are able to score an accuracy of 80%. Similarly, while analyzing models that are fine-tuned (Experiment 3), we observe the variance increases from 0.014 (while solving integer multiplication/division problems) to 0.068 (while solving decimal addition/subtraction problems) to 0.085 (while solving negative fractions problems). This trend suggests that models learn topics in an incremental manner, where the weaker models have been trained on certain simpler topics, while the better-performing models are more able to learn a wider range of topics. However, we note that this trend is not evident while observing ICL without CoT reasoning (Experiment 1). For this technique, we provide models with examples which only contain a problem and the corresponding answer. Although integer problems still have a low variance between models, the variance in all the other topics are mostly consistent at around 0.06. This is in contrast to the behavior when using the other techniques, where there is a more gradual increase in variance from the easier topics to the difficult topics. This could be explained by the lack of reasoning steps generated while employing ICL without CoT. This results in the answers generated being less accurate and the models may not have the opportunity to reason out the difficult problems, resulting in a worse accuracy even amongst the better-performing models.

To further verify the behavior of how models learn and illustrate the potential hierarchy in topic difficulties, we perform pairwise comparisons of some similar models and observe how they differ. 3 sets of comparisons are performed, representing models with a smaller capacity, moderately good models and strong-performing models respectively. Firstly, we observe the topic results of the Rho-1B model and TinyLlama. These models are both 1.1B parameter models, and Rho-1B weights were constructed by performing continual training on the TinyLlama model. Although there is a general increase in performance across topics, the largest accuracy increases come from topics such as ‘integer addition and subtraction’ (from 21.9% to 60.9%), ‘integer multiplication and division’ (from 27.3% to 62.6%) and ‘fraction addition and subtraction’ (from 0.0% to 56.0%). The accuracy of certain topics such as ‘negative fractions arithmetic’ only increase from 5.7% to 14.3%, or 4.0% to 28.0% for ‘rates and percentages’, or ‘rational numbers arithmetic’ from 11.1% to 22.2%. This represents an improvement in the fundamental topics when refining the small capacity models, but some of the more complicated topics remain a struggle.

We progress to comparing Mistral and MetaMath-Mistral. Both models are 7B parameter models which are built on the same architecture. MetaMath-Mistral has been trained to include a corpus with more mathematical formulas and context. In this situation, we observe that the ‘basic’ topics highlighted previously still continue to improve, with ‘integer addition and subtraction’ problems increasing to 73.4% in MetaMath-Mistral. However, some of the topics which are too challenging for the Rho model show marked improvements while generating responses with the Mistral models. For instance, the accuracy for ‘rates and percentages’ problems increases from 20.0% to 68.0%, while ‘rational number arithmetic’ improves from 22.2% to 55.6% accuracy. However, ‘negative fraction arithmetic’ remain a difficult topic, remaining at around 25% accuracy for both models.

To observe how model performances progress for the better-performing models, we closely analyze the results of Llama 3 and Llama 3.1. Both models are 8B parameter models built by Meta and feature the same architecture. They were released 3 months apart but show a distinct difference in performance. Both models improve upon the integer and decimal problems as compared to the Mistral models, but show minimal gains from Llama 3 to Llama 3.1. Instead, the largest gains come from topics such as ‘negative fraction arithmetic’ where the accuracy increases from 60.0% to 82.9%, and external topics which has an improvement by over 35% (from 64.7% to 100%). These topics are considered more challenging by the previous models and exhibited relatively small changes between the respective pairs of models.

Overall, this hints at a potential hierarchy in topic difficulty where foundational arithmetic skills are improved with smaller capacity models, while more complex topics show substantial gains only with stronger models. As model capacity and training data increases, models become more capable of handling advanced problem types, though the simpler topics benefit from improvements across the board. This progression suggests that smaller models tend to struggle predominantly with basic operations, but their enhancement leads to a focus on foundational skills, whereas larger models, after mastering these basics, can tackle more complex mathematical reasoning. This analysis highlights the value of a dataset that can perceive the current bottlenecks within mathematical reasoning so that we can focus on continued training and innovation of techniques for improved mathematical reasoning capabilities.

Table 17: Accuracy scores by topic per model while employing **Top**: ICL without CoT (Experiment 1); **Bottom**: Fine-tuning (Experiment 3)
The topic mean and variance for ICL without CoT excludes Vicuna as explained in Subsection C.5

Model	Integers (+ -)	Integers (* /)	Decimals (+ -)	Decimals (* /)	Fractions (+ -)	Fractions (* /)	Decimal to Frac	Rates and %	Exponents and $\sqrt{}$	Variables & Expressions	Linear Eqs	Negative Numbers	Negative Fractions	Mean	Variance	Coeff of Variation (CV)
Falcon	0.352	0.636	0.308	0.137	0.000	0.000	0.553	0.000	0.074	0.206	0.000	0.154	0.000	0.186	0.047	1.170
Gemma	0.453	0.748	0.692	0.397	0.000	0.010	0.342	0.120	0.389	0.318	0.136	0.333	0.029	0.305	0.059	0.795
Llama 3	0.953	0.899	0.936	0.699	0.240	0.423	0.500	0.400	0.704	0.794	0.379	0.795	0.257	0.614	0.067	0.420
Mistral	0.695	0.808	0.859	0.370	0.040	0.164	0.474	0.160	0.444	0.508	0.301	0.487	0.086	0.415	0.070	0.638
Phi-2	0.758	0.818	0.808	0.548	0.040	0.327	0.474	0.200	0.556	0.683	0.379	0.590	0.114	0.484	0.067	0.534
Qwen	0.977	0.950	0.936	0.781	0.440	0.673	0.684	0.440	0.778	0.825	0.544	0.692	0.571	0.715	0.033	0.255
TinyLlama	0.359	0.576	0.218	0.110	0.000	0.067	0.316	0.040	0.037	0.143	0.039	0.128	0.029	0.159	0.028	1.061
WizardMath	0.695	0.828	0.615	0.260	0.480	0.567	0.316	0.600	0.722	0.746	0.563	0.692	0.257	0.565	0.035	0.332
Vicuna	0.320	0.081	0.167	0.055	0.040	0.010	0.184	0.040	0.222	0.095	0.185	0.051	0.000	0.112	0.009	0.861
Zephyr	0.719	0.798	0.731	0.507	0.080	0.125	0.447	0.160	0.370	0.556	0.204	0.513	0.029	0.403	0.069	0.654
DeepSeekMath	0.898	0.960	0.897	0.849	0.800	0.914	0.842	0.960	1.000	0.937	0.922	0.974	0.857	0.909	0.004	0.065
Mathstral	0.781	0.869	0.731	0.630	0.240	0.433	0.605	0.240	0.630	0.619	0.515	0.615	0.343	0.558	0.039	0.352
MetaMath-Mistral	0.719	0.748	0.795	0.343	0.343	0.240	0.635	0.600	0.444	0.460	0.631	0.590	0.171	0.529	0.037	0.364
Rho	0.508	0.647	0.718	0.301	0.000	0.106	0.290	0.080	0.296	0.397	0.087	0.487	0.029	0.304	0.056	0.783
Llama 3.1	0.938	0.939	0.923	0.699	0.680	0.539	0.553	0.320	0.796	0.778	0.534	0.923	0.543	0.705	0.039	0.282
GPT3.5	0.984	0.950	1.000	0.890	0.320	0.654	0.553	0.760	0.722	0.857	0.621	0.846	0.486	0.742	0.043	0.280
GPT4o-mini	0.992	0.939	1.000	0.849	0.400	0.683	0.632	0.840	0.778	0.905	0.612	0.846	0.457	0.764	0.038	0.254
MEAN *	0.736	0.819	0.760	0.523	0.250	0.395	0.505	0.370	0.546	0.608	0.404	0.604	0.266			
VARIANCE *	0.048	0.015	0.051	0.067	0.065	0.082	0.022	0.091	0.073	0.062	0.067	0.063	0.065			

Model	Integers (+ -)	Integers (* /)	Decimals (+ -)	Decimals (* /)	Fractions (+ -)	Fractions (* /)	Decimal to Frac	Rates and %	Exponents and $\sqrt{}$	Variables & Expressions	Linear Eqs	Negative Numbers	Negative Fractions	Mean	Variance	Coeff of Variation (CV)
Falcon	0.547	0.596	0.487	0.178	0.000	0.394	0.316	0.080	0.148	0.270	0.136	0.282	0.114	0.273	0.035	0.687
Gemma	0.742	0.838	0.756	0.466	0.480	0.625	0.526	0.200	0.611	0.667	0.544	0.692	0.229	0.567	0.037	0.338
Llama 3	0.938	0.909	0.859	0.616	0.480	0.750	0.605	0.640	0.852	0.889	0.699	0.769	0.429	0.726	0.027	0.227
Mistral	0.680	0.778	0.359	0.370	0.320	0.769	0.526	0.440	0.815	0.746	0.660	0.692	0.314	0.575	0.037	0.333
Phi-2	0.813	0.889	0.808	0.575	0.680	0.779	0.684	0.600	0.982	0.857	0.748	0.949	0.457	0.755	0.023	0.202
Qwen	0.977	0.939	1.000	0.808	0.960	0.885	0.790	0.840	0.926	0.984	0.767	0.821	0.971	0.898	0.007	0.092
TinyLlama	0.344	0.616	0.180	0.069	0.000	0.269	0.395	0.000	0.056	0.079	0.019	0.077	0.029	0.164	0.036	1.154
WizardMath	0.898	0.899	0.897	0.575	0.520	0.769	0.711	0.840	0.833	0.889	0.738	0.769	0.629	0.767	0.016	0.167
Vicuna	0.734	0.778	0.564	0.301	0.080	0.539	0.526	0.120	0.389	0.556	0.243	0.359	0.057	0.404	0.056	0.588
Zephyr	0.836	0.849	0.539	0.343	0.440	0.683	0.526	0.560	0.685	0.714	0.631	0.718	0.200	0.594	0.035	0.316
MEAN	0.751	0.809	0.645	0.430	0.396	0.646	0.561	0.432	0.630	0.665	0.518	0.613	0.343			
VARIANCE	0.037	0.014	0.068	0.049	0.094	0.037	0.020	0.098	0.107	0.084	0.078	0.077	0.085			

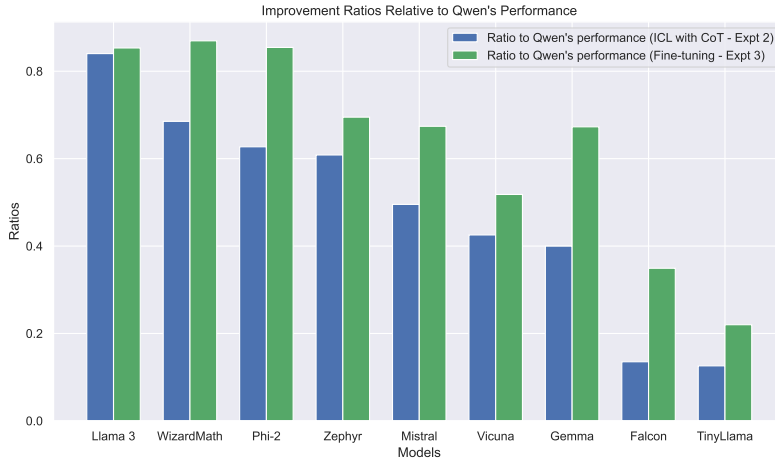


Figure 19: Ratio of each model’s performance relative to Qwen-7B model. The gap drastically decreases from using ICL with CoT (Experiment 2) to after performing fine-tuning (Experiment 3)

C.6 FINE-TUNING

A common technique to assist models in learning new distributions is by employing fine-tuning (Experiment 3). From Figure 19, it is evident that fine-tuning helps close the performance gap between models. As Qwen performs the best across the models we use for fine-tuning, we designate it as a benchmark model for this analysis. With this benchmark, we observe that each model’s relative performance to Qwen increases when we employ fine-tuning instead of in-context learning with chain-of-thought reasoning (ICL with CoT; Experiment 2). Notably, initially weaker models such as Gemma or Falcon substantially close the gap between their performance and the Qwen model performance.

To further analyze the benefits of fine-tuning (Experiment 3) compared to ICL with CoT (Experiment 2), we analyze how the model performances change between the two techniques (Table 18). For this analysis, we only extract the 10 models that are also fine-tuned and aggregate their performance while employing ICL with CoT. As stated earlier, we observe how fine-tuning helps to close the gap between the weaker models and the best model. This is reflected in every topic, where there is an increase in the mean accuracy ranging from 7.5% to 28.5%. However, there are some differences in how the performance of each topic is affected by fine-tuning.

Firstly, there are significant improvements in the mean accuracy for integer and decimal arithmetic, where we see a 16.1% improvement for integer multiplication and division and a 9.1% improvement for decimal addition and subtraction. The reduction in variance in these four topics suggest that although the models have different starting points, fine-tuning helps to bridge this gap and bring their performances on these simpler tasks to a more standardized level. This suggests that fine-tuning helps all models learn fundamental arithmetic concepts better, and serves as an effective method to make models of different capabilities and architectures have a more uniform performance in handling basic operations. These basic arithmetic tasks are relatively easier for models to learn and generalize when exposed to appropriate training data.

An interesting observation is how models react to fractions. On one hand, fraction problems involving multiplication and division demonstrate the largest improvement, increasing by 28.5%, while showing a reduction in variance amongst models. On the other hand, fractions involving addition and subtraction has one of the smaller improvements, only increasing by 8.4%. The slight increase in variance indicates that different models are learning how to handle fraction addition and subtraction in a different pace, potentially due to the architecture of the models or the data they were initially trained with. To understand why there exists a difference between these two topics, we can observe some of the responses generated by the models. From Figure 20, we see that models struggle with the conversion of denominators when adding or subtracting fractions, especially if there are more than 2 operands. On the other hand, multiplication and division of fractions can be seen as an

Table 18: Mean and variance of model performance in various topics. *Improvement in mean* represents the increase in accuracy when performing fine-tuning (Experiment 3) over using ICL with CoT (Experiment 2) while *ratio of variance* represents the proportional change in variance. A ratio of below 1 indicates a reduction in variance, while a ratio of above 1 demonstrates an increase in variance when using fine-tuning.

	Mean Acc (ICL CoT) (Expt 2)	Mean Acc (Fine-tuning) (Expt 3)	Improvement in Mean	Variance (ICL CoT) (Expt 2)	Variance (Fine-tuning) (Expt 3)	Ratio of Variance
Integers (+ -)	0.616	0.751	0.134	0.074	0.037	0.500
Integers (* /)	0.648	0.809	0.161	0.060	0.014	0.237
Decimals (+ -)	0.554	0.645	0.091	0.076	0.068	0.903
Decimals (* /)	0.355	0.430	0.075	0.067	0.049	0.732
Fractions (+ -)	0.312	0.396	0.084	0.087	0.094	1.084
Fractions (* /)	0.362	0.646	0.285	0.083	0.037	0.446
Decimal to Frac	0.418	0.561	0.142	0.037	0.020	0.549
Rates and %	0.360	0.432	0.072	0.076	0.098	1.289
Exponents and $\sqrt{\cdot}$	0.535	0.630	0.094	0.103	0.107	1.037
Variables & Expressions	0.554	0.665	0.111	0.086	0.084	0.982
Linear Equations	0.417	0.518	0.102	0.075	0.078	1.031
Negative Numbers	0.515	0.613	0.097	0.073	0.077	1.048
Negative Fractions	0.271	0.343	0.071	0.063	0.085	1.352

Question: Solve: $((3)/(4)) + ((1)/(6)) - ((2)/(3)) = ?$

Phi-2 Response: (Incorrect)

To solve this equation, we need to first find the common denominator of the fractions. The common denominator is 12.

Next, we need to convert each fraction to have a denominator of 12.

$((3)/(4))$ becomes $((9)/(12))$ and $((1)/(6))$ becomes $((2)/(12))$.

$((2)/(3))$ remains the same.

Now we can add and subtract the fractions:

$((9)/(12)) + ((2)/(12)) - ((2)/(12)) = ((11)/(12))$.

Therefore, $((3)/(4)) + ((1)/(6)) - ((2)/(3)) = ((11)/(12))$.

Output: $((11)/(12))$

Gemma Response: (Incorrect)

To solve, we need to find a common denominator. The least common denominator is 12. So, we need to multiply $3/4$ by 3 and $1/6$ by 2 to get $9/12$ and $2/6$. Then, we can rewrite the equation as $9/12 + 1/6 - 2/6 = 11/12$. [ANSWER] $11/12$

Qwen Response: (Correct)

First, find a common denominator for the fractions, which is 12 in this case. Convert each fraction:

$((3)/(4))$ becomes $((3*3)/(4*3)) = ((9)/(12))$

$((1)/(6))$ becomes $((1*2)/(6*2)) = ((2)/(12))$

$((2)/(3))$ becomes $((2*4)/(3*4)) = ((8)/(12))$

Now, perform the operations: $((9)/(12)) + ((2)/(12)) - ((8)/(12)) = ((9+2-8)/(12)) = ((3)/(12))$

Simplify the fraction: $((3)/(12)) = ((1)/(4))$

So, the answer is $((1)/(4))$. [ANSWER] $((1)/(4))$

Figure 20: Sample model responses when answering addition and subtraction fraction problems

extension of multiplication and division of integers, and hence models are better able to learn the reasoning required.

For some topics, models continue to have difficulties even after fine-tuning. For example, ‘rates and percentages’ and ‘negative fractions arithmetic’ show some improvement in accuracy, but a large increase in variance. This suggests that while fine-tuning helps improve the models’ overall performance on these tasks, it is not an even improvement across all models. Some models might be adapting better than others, leading to an increase in the variance. This further supports the earlier discussion in Subsection C.5, where we suggest the existence of hierarchy amongst the topics, which results in barriers for some of the less capable models in answering accurately in specific topics. This indicates that there may be challenges that are not easily resolved by only employing fine-tuning. The KMX dataset serves as a stepping stone towards exploring other techniques or architectural changes that can help models to improve their ability to solve these challenging topics.

In summary, fine-tuning generally leads to improvements in accuracy across most models, especially for fundamental arithmetic topics, but the gain in performance varies for more complex problems. The reduction in variance for simple tasks like integer operations and decimal arithmetic indicates that fine-tuning helps different models perform these tasks more uniformly, while the increased variance in topics like ‘rates and percentages’ suggests underlying difficulties for certain models. This means that while fine-tuning may be beneficial, it may not fully address the deeper reasoning challenges of all models. The KMX dataset can assist in addressing these challenges by providing a more diverse training corpus. By having different types of questions and topics, the dataset can aid models in generalizing better which can then result in less variability and improved accuracy across the models.

Table 19: Model accuracy when performing greedy decoding and self-consistency (SC). A decrease in *gain using SC* implies a decrease in accuracy when self-consistency is employed. An asterisk (*) denotes that the difference is significant at 5% significance level.

Left: ICL with CoT (Experiment 2); **Right:** Fine-tuning (Experiment 3)

Model	Greedy Decoding	SC	Gain using SC	Model	Greedy Decoding	SC	Gain using SC
Falcon	0.122	0.116	-0.005	Falcon	0.314	0.236	-0.079*
Gemma	0.359	0.315	-0.044*	Gemma	0.605	0.524	-0.082*
Llama 3	0.755	0.790	0.035*	Llama 3	0.768	0.813	0.045*
Mistral	0.445	0.447	0.002	Mistral	0.606	0.640	0.034
Phi-2	0.563	0.589	0.025	Phi-2	0.769	0.744	-0.025*
Qwen	0.898	0.918	0.020	Qwen	0.900	0.920	0.021
TinyLlama	0.113	0.104	-0.009	TinyLlama	0.198	0.165	-0.033
WizardMath	0.616	0.722	0.106*	WizardMath	0.782	0.791	0.008
Vicuna	0.382	0.440	0.058*	Vicuna	0.466	0.494	0.028
Zephyr	0.547	0.602	0.056*	Zephyr	0.625	0.607	-0.018
DeepSeekMath	0.888	0.936	0.048*				
Mathstral	0.772	0.813	0.041				
MetaMath-Mistral	0.642	0.720	0.079*				
Rho	0.483	0.416	-0.067*				
Llama 3.1	0.858	0.892	0.035*				
GPT3.5	0.870	0.902	0.031*				
GPT4o-mini	0.956	0.948	-0.008				

C.7 SELF-CONSISTENCY

As an add-on to language model techniques, self-consistency (SC) involves performing inference multiple times (five in our experiments) with a high temperature to introduce variability, and then selecting the most frequent solution as the final answer. This technique has been shown to improve performance in some tasks. At a macro level, we want to determine whether applying self-consistency is universally beneficial or detrimental to mathematical reasoning across different models. If we are able to apply self-consistency in a way that it improves the performances of models in a few topics, and has negligible impact on the other topics, it would signify that self-consistency could be applied without negative consequence. To test this theory, we apply a t-test on the topic accuracies for greedy decoding (GD) and self-consistency. If the t-test indicates a significant difference in performance, this would confirm that self-consistency has either a positive or negative impact on the model’s ability to solve mathematical tasks across all topics.

When looking at the ICL with CoT (Experiment 2) scenario (Table 19), we observe that close to half of the models have a marginal effect from employing self-consistency. Amongst the remaining models, 8 of the models demonstrate significant (at 5% level of significance) gains in performance by using self-consistency. For example, the accuracy of the Zephyr model increases from 54.7% to 60.2%, and DeepSeekMath also has a performance gain from 88.8% to 93.6%. However, Gemma and Rho both show a decline in performance (a drop of 4.4% and 6.7% respectively), favouring greedy decoding instead. This is a noteworthy observation as it implies that self-consistency can even be detrimental and we need to be aware of how we apply it.

This observation is further supported when we compare the results of the models which underwent fine-tuning (Experiment 3). Out of the 10 models, 6 models exhibit mixed results towards self-consistency, while 3 models favour greedy decoding at a 5% significance level. Self-consistency seems to be clearly favourable for the Llama 3 model, where the accuracy increases from 76.8% to 81.3%.

Based on preliminary analysis, there are some possible explanations for the varied effectiveness of self-consistency. We observe that greedy decoding is usually favourable for models that do not have a high accuracy. This may represent a limitation in the model’s ability to generate appropriate solutions to the math problems. By introducing a high temperature to the generation process, it further amplifies these models’ deficiencies and does not aid in providing meaningful alternative reasoning

Table 20: Accuracy difference when using self-consistency (SC) for specific tasks. An asterisk(*) denotes a statistical significant result across the models at a 5% significance level

Task	Improvement using SC (ICL CoT - Expt 2)	Improvement using SC (Fine-tuning - Expt 3)
Integer (* /)	0.0493*	0.0020
Decimal (* /)	0.0363*	0.0096
Exponents and $\sqrt{\cdot}$	-0.0033	-0.0518*
Negative Numbers	0.0452*	-0.0128
Negative Fractions	-0.0218	-0.0914*
One OpType Expr	0.0466*	0.0064
Multi Op Expr	-0.0115	-0.0935*
Multi-step Algorithm	0.0328*	0.0163

paths, instead adding more noise compared to an answer that is generated deterministically. Hence, this may result in self-consistency being detrimental for these models. On the contrary, many of the better-performing models benefit from self-consistency when we use in-context learning. These models can make use of their capabilities to generate various responses that may help towards making a decision of an accurate answer. However, after fine-tuning these models with the KMX dataset, the models become more confident in solving the questions that are present in the test set. When we apply greedy decoding, it helps the models utilise this gained knowledge through fine-tuning to generate accurate answers. In this case, self-consistency may hinder with this learned information and hence limit the effectiveness of employing self-consistency.

Given that self-consistency helps some models and hurts others, it is worth looking into whether specific tasks are more or less suited for self-consistency. We analyze the performance of the models within each question type and topic, performing a statistical t-test using each model’s accuracy to determine if the usage of self-consistency has a general effect on models for a specific task (Table 20). In the case of question types involving multi-step algorithms, we observe an improvement in using self-consistency in both situations, hinting at the idea that further researching about self-consistency in this situation may yield further results.

Similar to the results observed when analyzing the models, we notice that some tasks benefit from self-consistency while applying ICL with CoT (Experiment 2), but produce mixed results when applying fine-tuning (Experiment 3). For instance, self-consistency has a nearly 5% increase in performance for multiplication and division integer problems while employing in-context learning, but a negligible effect for the fine-tuned models. In some cases, the two techniques exhibit opposing behaviors when self-consistency is applied. For problems involving negative numbers, models applied with in-context learning benefit by a mean improvement of 4.52% when using self-consistency over greedy decoding, but the fine-tuned models favor greedy decoding by 1.28%.

However, we note that there are situations where greedy decoding is the preferred approach for answer generation. In solving problems involving negative fraction arithmetic or problems categorized as ‘expressions with multiple operation types’, self-consistency performs worse than greedy decoding. These results contradict broader claims in the literature that self-consistency is a generally superior method.

This variability highlights that while self-consistency can help improve accuracy in certain situations, its application must be contextualized to the specific model and task. The nuances observed in our experiments show that commonly used techniques like self-consistency require deeper analysis to determine when and why they work. This further motivates the KMX dataset in providing a fine-grained analysis framework to explore model behaviors across different mathematical topics and question types. Careful consideration is needed before broadly implementing such techniques as their benefits may not universal.

Table 21: Accuracy of models while employing fine-tuning (Experiment 3), calculator without pre-training (Experiment 4), and calculator with pre-training (Experiment 5). Each experiment is conducted with generating via greedy decoding (GD) and self-consistency (SC).

Bold: Best performing technique for the model; *italicise*: Worst performing technique for the model; MEAN*: Mean accuracy excluding Falcon and Mistral due to their inability to generate meaningful outputs after pre-training

Model	Fine-tuning (Expt 3)		Calculator (Expt 4)		Calc w/ pre (Expt 5)	
	GD	SC	GD	SC	GD	SC
Falcon	0.314	0.236	0.312	0.266	<i>0.140</i>	0.249
Gemma	0.605	<i>0.524</i>	0.659	0.627	0.672	0.687
Llama 3	0.768	0.813	<i>0.715</i>	0.776	0.751	0.791
Mistral	0.606	0.640	0.589	0.680	<i>0.270</i>	0.429
Phi-2	0.769	0.744	0.738	0.740	0.744	0.688
Qwen	0.900	0.920	0.907	0.922	0.887	0.918
TinyLlama	0.198	<i>0.165</i>	0.382	0.340	0.406	0.390
WizardMath	0.782	0.791	0.800	0.809	0.816	0.814
Vicuna	<i>0.466</i>	0.494	0.577	0.631	0.595	0.676
Zephyr	0.625	<i>0.607</i>	0.646	0.662	0.639	0.646
MEAN	0.603	0.593	0.632	0.645	<i>0.592</i>	0.629
MEAN*	0.639	<i>0.632</i>	0.678	0.688	0.689	0.701

C.8 EXTERNAL CALCULATOR

External solvers are also an option for mathematical reasoning problems. In this experiment, we fine-tune models to recognize special tokens marking the start and end of calculations, allowing SymPy (Meurer et al., 2017) to evaluate expressions during inference. This aims to reduce arithmetic errors. While experimenting with external calculators, we attempt two different techniques. In the first experiment, we fine-tune the models directly on the training data with the calculator tags (Calculator; Experiment 4), while in the second experiment, we add a pre-training dataset to familiarize the model with the newly added tokens, before fine-tuning with the actual KMX data (Calculator with pre-training; Experiment 5). We run each experiment using greedy decoding as well as self-consistency during the generation process.

Looking at the average accuracy across all questions (Table 21), the impact of the external calculator is very polarizing. 3 of the models perform best without any additional calculator tags (Experiment 3), 3 of the models perform best when employing the calculator without the pre-training dataset (Experiment 4), and the remaining 4 models perform the best when employing the calculator with the pre-training dataset (Experiment 5). We observe that for the Falcon and Mistral models, they do not respond well to the additional pre-training dataset and start to produce nonsensical outputs, resulting in performances worse than without the pre-training dataset or without using an external calculator at all. Excluding these two models, we observe that the average accuracy while using the calculator with pre-training is 70.1%, while fine-tuning only achieves a 63.2% accuracy. Therefore, there is promise to this technique, although we need to be aware how different models may react to such additional inclusions.

To provide a better understanding of the potential of using an external solver, we make use of the KMX dataset to perform more granular analysis. For instance, when solving integer addition and subtraction problems, using the calculator with the pre-training dataset and self-consistency yields the best performance in 7 out of the 10 models. The remaining 3 models are Falcon, Mistral (which do not learn the tags after pre-training the models) and Qwen (which got one question wrong as compared to without pre-training where it scored 100%).

To conduct further comparisons between the use of calculator with pre-training (Experiment 5) and the original fine-tuning (Experiment 3), we first exclude the two models that are unable to produce meaningful results after the pre-training with the new tokenization. Our experiments show that incorporating the external calculator result in an improvement in handling expressions with one type of operation (1 Op Expr), and multiple types of operations (Multi Op Expr). 6 out of 8 models

Table 22: Accuracy difference when using an external calculator with pre-training (Experiment 5). **Bolded** values represent improvements when using an external calculator

Model	1 Op Expr	1 Op MWP	Multi Op Expr	Algorithms
Gemma	0.279	0.074	0.152	-0.023
Llama 3	0.017	0.024	0.087	0.023
Phi-2	-0.010	-0.037	0.22	0.047
Qwen	-0.002	-0.012	0.022	-0.093
TinyLlama	0.442	0.235	0.022	0
WizardMath	0.083	-0.037	0.152	-0.070
Vicuna	0.240	-0.025	0.478	0.047
Zephyr	0.075	-0.012	0.087	-0.093

display improvements with the external calculator when solving ‘1 Op Expr’, with the remaining showing a negligible decrease, while all 8 models improve their performance in solving ‘Multi Op Expr’ (Table 22). However, the technique is less effective for math word problems with one type of operation (1 Op MWP) and multi-step algorithms, where improvements are in only 3 models each. This suggests that the benefits of using an external calculator are not universal and can potentially lead to performance drops in certain problem types.

A major challenge with employing the calculator is the additional dimensions that are present. For instance, when analyzing integer multiplication and division problems, Gemma performs the best when employing the calculator with pre-training with self-consistency (86.9% accuracy), but performs the worst when employing the same technique but with greedy decoding instead (76.8% accuracy). On the contrary, Phi-2 has a performance of 86.8% with greedy decoding but only 64.7% accuracy when generating outputs using self-consistency. Therefore, although self-consistency seems to be useful on an aggregated level when paired with a calculator, we need to be aware of the individual situations where this may not hold true.

In summary, the use of an external calculator has the potential to be useful in assisting models to solve mathematical problems. The inclusion of a pre-training dataset is largely beneficial, but can potentially interfere with some models’ ability to generate usable output. The results show the promise in incorporating external solvers, but further support the message that fine-tuning techniques must be carefully tailored to the characteristics of the dataset and model in question. It is beneficial to use the KMX dataset to break down the performances of models so we can progress towards understanding how models learn and what adjustments we should make to take advantage of their strengths.

C.9 IDENTIFYING THE EXERCISE NAME

We fine-tune models to first identify the exercise name before solving the problem (ID Exercise; Experiment 6), with the aim of investigating whether a simple inclusion of this additional dimension from the KMX dataset could offer valuable insights for improving model performance. This idea stems from analogical learning (Hu et al., 2023; Webb et al., 2023; Yasunaga et al., 2024) where we are first trying to get the model to categorize the problem it is solving and then generate output based on its prior interactions with problems of the same type. The accuracy of the categorization is not a focus in this method as there could be different ways to categorize a question, and we want models to take advantage of this to approach the problem from different directions. This is aligned with our results that show that this technique performs well when coupled with self-consistency. With self-consistency, models are able to identify different approaches and reach a consensus based on the different reasoning paths. As a whole, we observe that employing this technique only reduces the accuracy of the Llama 3 model, and improves the accuracy of 8 models from 1.2% to 10.6% (Table 23).

To comprehensively analyze the impact of this technique, we utilize the KMX dataset to break down the problems into their respective topics and question types (Table 24). By evaluating the mean difference in accuracy across 10 models for each category, we aim to determine whether the addition of exercise names provides a meaningful improvement in specific mathematical reasoning

Table 23: Model accuracy when fine-tuning with and without identifying the exercise name using self-consistency. A positive difference denotes an improvement when fine-tuning with the exercise name

Model	Fine-tuning (Expt 3)	ID Exercise (Expt 6)	Difference
Falcon	0.236	0.341	0.106
Gemma	0.524	0.592	0.068
Llama 3	0.813	0.758	-0.055
Mistral	0.640	0.664	0.024
Phi-2	0.744	0.773	0.029
Qwen	0.920	0.932	0.012
TinyLlama	0.165	0.198	0.033
WizardMath	0.791	0.833	0.042
Vicuna	0.494	0.494	0.000
Zephyr	0.607	0.629	0.022

tasks. In some difficult topics, models exhibit a significant gain when being prompted to write the exercise name. For example, the improvement for decimal addition and subtraction is statistically significant (p-value of 0.040), and the majority of the models show a marked improvement, with the two models that worsen having a change of only around 2% each. Similarly, solving problems involving negative fractions arithmetic results in an average of 8.6% increase in accuracy, and is significantly better than when excluding the exercise name from the fine-tuning process (p-value of 0.028). However, there are some topics that produce varied results when applying this technique. For instance, half of the models exhibit a decrease in accuracy by over 1% when solving multiplication and division fraction problems.

When split by question types, fine-tuning with exercise names have similar variations across different question types. For the tasks that involve the conversion of numbers between formats, such as ‘decimal to fraction conversion’ and ‘converting scientific notation’, the models show performance gains (9.6% and 8.8% increase in mean accuracy respectively) in a majority of the models. Conversely, for simpler problem types, such as solving ‘expressions with one operation type’ (2.4% increase in mean accuracy) or ‘quotient-remainder’ tasks (0% change in mean accuracy), the identification of exercise names may be extra noise that does not provide useful context to aid the generation process.

These results indicate that fine-tuning with exercise names is a promising tool in improving model accuracy, particularly for tasks that may be more complicated. However, the technique is less effective for some basic arithmetic tasks, especially involving multiplication or division of decimals and fractions. While further research is required, these findings indicate that even simple exercise-level annotations offer an encouraging direction towards improving model training and evaluation. The detailed labeling of the KMX dataset provides an avenue for future work to refine the use of exercise names or other additional information, potentially tailoring the technique to areas where there is substantial gains while avoiding its usage in problem types where performance declines are observed.

Table 24: Change in mean accuracy over 10 models when fine-tuning with exercise name (Experiment 6). A positive *Mean Diff* denotes an improvement when fine-tuning with exercise name compared to the original fine-tuning (Experiment 3) when averaging all 10 models. *Improved*, *No Change*, *Decreased* denote the number of models (out of 10) that show an improvement of at least 1%, a change between -1% to 1%, and a decrease in accuracy by over 1% respectively.

(a) By Topic

Topic	Mean Diff	Improved	No Change	Decreased
Integers (+ -)	0.012	5	2	3
Integers (* /)	0.017	4	2	4
Integer (Multi-Op)	0.028	4	3	3
Decimals (+ -)	0.067	7	1	2
Decimals (* /)	-0.008	6	0	4
Fractions (+ -)	0.044	6	3	1
Fractions (* /)	0.004	4	1	5
Decimal to Frac	0.071	7	2	1
Rational Numbers	0.044	5	4	1
Rates and %	0.020	7	0	3
Exponents and $\sqrt{\cdot}$	0.035	5	2	3
Factors	0.046	4	4	2
Variables & Expressions	0.029	5	1	4
Linear Equations	0.027	6	2	2
Negative Numbers	0.023	6	1	3
Negative Fractions	0.086	7	2	1
External Topics	0.047	5	2	3
Scientific Notation	0.048	8	1	1

(b) By Question Type

Question Type	Mean Diff	Improved	No Change	Decreased
One OpType Expr	0.024	5	4	1
Two OpType Expr	0.041	9	1	0
Multi OpType Expr	0.015	5	2	3
One OpType MWP	0.004	5	2	3
Multi OpType MWP	0.034	6	2	2
Quotient-Remainder	0.000	4	4	2
Convert Decimal Fraction	0.096	7	2	1
Write Expression	0.036	7	0	3
Expressions w/ Variables	0.012	5	1	4
MWP w/ Variables	0.062	7	1	2
Multi-step Algorithm	0.026	5	2	3
Convert Scientific Notation	0.088	7	2	1